Table of Contents

1. Informative Part . . . 2
   1.1. Team . . . 2
   1.2. Current Situation, Needs, and Ideas . . . 4
   1.3. Scope, Span, and Synopsis . . . 8
   1.4. Foundational Activities for System Development . . . 10
   1.5. Derived Goals . . . 12
2. Descriptive Part . . . 16
   2.1. Domain Description . . . 16
   2.2. Requirements . . . 33
   2.3. Implemented End-to-End Features . . . 46
3. Analytic Part . . . 55
   3.1. Concept Analysis . . . 55
   3.2. Domain Analysis and Concept Formation Pipeline . . . 69
   3.3. Requirements Inconsistency Analysis . . . 74
   3.4. Validation and Verification . . . 80
   3.5. Requirements Inspection Checklist (Defect-Driven) . . . 88
4. Specific Topics . . . 91
   4.1. Stakeholder Identification . . . 91
   4.2. Personas . . . 94
   4.3. Traceability . . . 97
   4.4. Domain Requirements Acquisition . . . 100
   4.5. Inconsistency Management . . . 104

NOTE

Milestone 2 Change Tracking: To assist in the evaluation process, all new additions, features, and major revisions introduced during Milestone 2 are highlighted in yellow.

Example: This indicates new or updated content for Milestone 2.

1. Informative Part

1.1. Team

For Milestone 2, the Digital Home Inventory team is organized into two project managers, a cross-functional Documentation Team, and four feature teams: Core Inventory, Alerts & Restock, Sharing & Budget, and Reports & Insights. This structure reflects how the milestone work is actually distributed in the tables below: each feature team is responsible for advancing its area through coordinated design, backend, documentation, testing, and implementation work, while the Documentation Team supports milestone-wide consistency, integration of changes, and delivery of the official project documentation. All listed team members are considered partners because they have direct responsibility for shaping Milestone 2 outcomes, including analysis, requirements refinement, implementation, validation and verification, and documentation. They contribute to project decisions and deliverables rather than participating only as future end users of the system-to-be. No external participants are treated as formal partners for this milestone, since interviewed students and other domain informants mainly provide evidence and feedback, but do not hold recurring responsibility for project decisions, implementation, or milestone deliverables. If an external participant later takes on an ongoing role such as recurring requirements validation, architectural review, or milestone-level approval support, that role and its responsibilities will be documented here. The tables below identify each group and team lead, and the Tasks column records the associated GitHub issues used to trace each member's contributions.

Managers:

Student	GitHub username	Tasks
Kaysha L Pagan-Lopez	@Kay9876	439, #441
Luis J Cruz-Cruz	@LuisJCruz	453, #455

Documentation Team:

Student	GitHub username	Tasks
Alondra S. Arce Perea (Team Lead)	@alondra-arce	13, #113, #150, #177 #233, #238, #244, #250, #345, #398, #406, #468, #470, #471, #472, #514
Januel E. Torres Marquez (Team Lead)	@jankii03	52, #135, #136, #137, #324, #325, #326, #327, #345, #406, #489, #490

Team 1: Core Inventory:

Student	GitHub username	Tasks
Kevin Gómez (Team Lead)	@kevgom018	11, #12, #14, #22, #34, #107, #166, #271, #272, #273, #274, #275 #282, #341, #402, #435
Adriana Figueroa (Designer)	@AdrianaFigueroa9	19, #22, #34, #51, #93, #94, #107, #176, #278, #282, #288, #290, #330
Devlin Hahn (Designer)	@devlin-hahn04	38, #39, #41, #145, #280, #287, #393, #431

Student	GitHub username	Tasks
Jorge De León (Designer)	@JorgeDeLeonOrama	17, #116, #126, #127, #317, #318, #319 #358, #409
Kian-Robert Ramos Vega (Backend)	@kian-robert	67, #108, #109, #131, #279, #372, #376, #438
Lucas Matos Datiz (Backend)	@L-A-M-D	82, #231, #249, #264, #267, #391, #515, #516, #519
Jorge Rivera (Documentation)	@JorgeRivera94	15, #122, #276, #277 #382

Team 2: Alerts & Restock:

Student	GitHub username	Tasks
Solimar Cruz Hernandez (Team Lead)	@Solimar-Cruz	22, #30, #32, #33, #34, #107, #175, #282, #297, #298, #299, #328, #329, #342 #403
Ronaldo E. Flores Nazario (Designer)	@ronaldoflores9	20, #42, #84, #320, #321, #322, #323
Eliel Hernandez Gonzalez (Designer)	@eliel-hernandez	25, #26, #153, #303 #304, #305 #306
Kemuel Cubero Rosario (Backend)	@Kemuel05	23, #146, #147, #148, #313 #314, #315, #351, #352
Aryam Z Diaz Torres (Backend)	@aryamdiaz	24, #112, #180, #181, #291, #292, ##293, #294, #378, #379
Kevin Lara Rodríguez (Documentation)	@kevinlara1	87, #205, #206, #207, #399
Reinaldo J. Martinez Morales (Documentation)	@reinaldo-martinez1	27, #28, #119, #199, #295, #296, #427, #428

Team 3: Sharing & Budget:

Student	GitHub username	Tasks
Andrea Segarra (Team Lead)	@andreasegarra	22, #34, #35, #107, #132, #134, #179, #282, #343, #404, #483
Daniella Melero (Designer)	@daniellameleroo	40, #44, #45, #48, #310, #331 #348, #349, #350
Natalia S Vera Rivera (Designer)	@nataliavera6	21, #102, #103, #104, #106, #309, #354, #355, #356, #400
Fernando Castro (Backend)	@fernan-castro	71, #168, #173, #198, #308, #448, #449

Student	GitHub username	Tasks
Ernesto S. Soto Rivera	@ernestosoto3	31, #98, #99, #100, #307, #423, #424, #451
Fabiola Torres (Documentation)	@FabiolaZTorres	46, #95, #96, #101, #362, #363, #388, #389, #429
Yamilette Alemany (Documentation)	@yamilette-alemany	18, #58, #59, #60, #339 #340, #416, #417, #418

Team 4: Reports & Insights:

Student	GitHub username	Tasks
Fabian Velez (Team Lead)	@FabianVelezOcasio	22, #34, #107, #138, #139, #157, #178, #282, #344, #405, #420, #421, #422
Ian Rivera (Designer)	@Programian	29, #115, #118, #141, #300, #311, #334, #335, #337
Jayden Sánchez (Designer)	@jaydensanchez12	56, #129, #130, #332, #333, #426
Jorge Peralta (Backend)	@PeraltaJorge9128	43, #143, #159, #289, #316
Samarys Barreiro Melendez (Backend)	@SamarysB	36, #111, #113, #155, #156, #408, #433, #434, #344, #480
Gerardo Ruiz (Backend)	@Gerardoruiz1	55, #201, #261, #263, #312, #336, #466
Nicolás Rivera Batista (Documentation)	@nicolasrivera25	16, #193, #196
Aliana Figueroa (Documentation)	@alianasofiaaa	#

1.2. Current Situation, Needs, and Ideas

1.2.1. Current Situation

Students living independently often lose track of household items such as groceries, cleaning supplies, toiletries, and sometimes medicine. Tracking is usually ad-hoc (memory, notes, receipts, or messages), so information becomes outdated quickly. As a result, students may buy duplicates, run out of essentials unexpectedly, and sometimes discover expired items during cleanup.

In shared households, the problem is worse: roommates lack visibility into what is already available, which increases duplicate purchases and makes budgeting harder. In many student households there is no consistent shared method for tracking items across people and time; tracking is ad-hoc (memory, notes, messages), so information becomes stale quickly.

1.2.1.1. Domain context

Students and young adults living independently often struggle to keep track of household items, shared supplies, and personal belongings. While several home inventory applications exist, most are designed for insurance documentation or business inventory management rather than

lightweight everyday use.

As a result, many users rely on memory, notes, or informal tracking methods, which leads to forgotten purchases, duplicate buying, and poor visibility into shared household resources. Most tools students try (notes, basic lists, or heavier inventory apps) are either too informal to stay accurate over time or too cumbersome for everyday tracking, so shared inventory information quickly becomes unreliable.

1.2.1.2. Who is involved (actors)

- Primary actors: Students and young adults managing personal household items
- Secondary actors: Roommates or cohabitants sharing supplies
- Organizations/systems involved: Existing home inventory mobile applications (e.g., Sortly, PantryCheck)

1.2.1.3. Current workflow (as-is)

1. User purchases or owns household items
2. User optionally records items manually in notes or an inventory app
3. Inventory is rarely updated consistently over time
4. User forgets what items are already available
5. User repurchases duplicate or unnecessary items
6. In shared households, roommates lack visibility into available supplies

1.2.1.4. Pain points

Pain point	Who experiences it	Evidence / example
Too much of a hassle when adding items	Students and casual users	User reviews have reported that manual item entry consumes too much time and discourages the continued use of the software.
Inventory becomes outdated quickly	Almost all users	Many users abandon inventory apps after initial setup due to maintainance taking too much time.
Excessive or poorly timed notifications	Mobile users	Reviews indicate that too many notifications pop up in other applications, leading users to disabling alerts, one of the main purposes of said apps.
Pricing perceived as too high for casual use	Students and young adults	Premium tiers are often designed for business users rather than individuals, but lock features that could come in handy for individual users.

Pain point	Who experiences it	Evidence / example
Lack of effective shared household inventory support	Roommates and cohabitants	Most existing apps provide limited or no real-time collaboration features, which makes it difficult for multiple household members to maintain a single accurate inventory. This can lead to miscommunication or duplicate purchases for the house/dorm.

1.2.1.5. Consequences

- Time/cost impact: Users and students waste money on duplicate purchases and spend extra time checking inventory manually using written lists or other tracking software.
- Quality/accuracy impact: When tracking relies on tools that are abandoned or partially used (e.g., because advanced features are unavailable or too cumbersome), the record becomes incomplete and therefore untrusted.
- User experience impact: Frustration leads to abandonment of existing apps.
- Organizational impact: Shared households experience coordination inefficiencies.

1.2.1.6. Assumptions and constraints (as-is)

- Assumption: Target users prefer low-effort mobile workflows.
- Assumption: Users are more likely to maintain inventory if entry is fast and simple.
- Constraint: Users have limited patience for manual data entry.
- Constraint: The solution must work well on mobile devices with intermittent connectivity.

1.2.2. Needs

The needs below describe what students living independently and their roommates must be able to accomplish to manage household essentials reliably. These needs are expressed as outcomes and capabilities (what stakeholders need to achieve), independent of any specific system-to-be.

Needs by actor group

1.2.2.1. Actor group: Students and young adults

- Need N1: Minimize the effort and time required to keep track of household items
- Need N2: Maintain an accurate understanding of what items are available as they are consumed
- Need N3: Determine whether an item is already available before making a purchase

1.2.2.2. Actor group: Roommates / Cohabitants

- Need N4: Maintain a shared understanding of what items are available to the household
- Need N5: Coordinate responsibility for replenishing shared items to avoid duplicate or missed

purchases

1.2.2.3. Prioritized needs

ID	Need statement	Priority	Rationale
N1	Users must be able to record items in seconds	High	Addresses high-friction item entry reported in user feedback
N2	Users must maintain an up-to-date inventory with low ongoing effort	High	Mitigates abandonment caused by needing high maintainance
N3	Users must quickly verify existing items before purchasing	High	Reduces duplicate purchases and wasted spending
N4	Household members must share inventory visibility	Medium	Improves coordination in shared living environments
N5	Household members must coordinate replenishment of shared items to avoid duplicate or missed purchases	Medium	Reduces miscommunication and duplicate spending in shared living environments

1.2.2.4. Quality Needs (Non-Functional)

- Usability: Household members need to update item tracking with minimal effort and without training.
- Reliability: Household members need to trust that the tracked inventory matches what is actually available.
- Performance: Household members need inventory information quickly enough to use it while shopping or planning.
- Availability: Household members need tracking to remain usable even when connectivity is unreliable.
- Security/Privacy: Household members need confidence that household data is only visible to intended people.
- Scalability: Household members need the experience to remain smooth as more items, households, and members are added.

1.2.3. Ideas

Outline candidate solution directions. Ideas are not commitments; they are options to address the needs. The idea below focuses on solution directions and capabilities; implementation technology choices are deferred to later sections.

1.2.3.1. Lightweight Smart Inventory

- Core concept: A lightweight household inventory tool optimized for very fast item capture and shared visibility in student households, so records stay current with minimal effort.

- Needs addressed: N1, N2, N3, N4, N5
- Key capabilities:
  - Fast item capture with minimal information required at the moment of entry
  - Shared household view of what items are available and who last updated them
  - Quick search and filtering to check item availability
  - Optional prompts to review items that may be running low or expiring (kept low-noise)
  - Works during temporary connectivity gaps and reconciles changes when connectivity returns
- Expected benefits:
  - Reduces the hassle of item entry
  - Improves long-term inventory accuracy
  - Prevents duplicate purchases
  - Enables smooth roommate coordination
  - Reduces the chance that people disable reminders by keeping prompts minimal and relevant
- Risks / unknowns:
  - Users may still abandon after initial novelty wears off
  - Conflicting updates when multiple household members change the same item list around the same time
  - Temporary connectivity gaps may lead to mismatched inventories until updates reconcile
  - User adoption of shared features may vary
- How we will validate this idea:
  - Run quick usability sessions to see whether people can capture an item fast enough during realistic moments (e.g., after shopping)
  - Track weekly retention during pilot testing
  - Test with small shared households (2-4 users)
  - Observe whether optional prompts are perceived as helpful or disruptive (and whether people disable them)

1.3. Scope, Span, and Synopsis

1.3.1. Scope and Span

Scope

The project's scope is to carry out a software engineering project in the domain of student household management. The project will investigate how students living independently (alone or with roommates) keep track of household items and spending, and it will produce a system-to-be plus the supporting deliverables needed to build it responsibly (domain description, requirements,

architecture/design, implementation, validation/verification, and documentation). The project targets common pain points caused by relying on memory and informal tracking, and it will support both individual and shared-household workflows. The project will encompass several key areas:

- Domain Engineering: Analyze how students currently manage household items, including common breakdowns such as forgetting what is available, inconsistent updates, and poor roommate coordination. Identify the major actors (students living alone, students living with roommates/cohabitants) and the domain concepts (items, categories, locations/rooms, quantities, thresholds, expiration dates, shared responsibility).
- Requirements Engineering: Define system requirements that support inventory accuracy, fast item updates, and coordination. Requirements will cover core inventory maintenance, item organization, information lookup before purchasing, shared-household coordination, and budget awareness, while also capturing constraints such as busy schedules, low tolerance for complex setup, and the need for privacy controls.
- Software Architecture: Develop a high-level architecture that supports mobile use, persistent storage, and shared-household data synchronization. The architecture will identify major components and their responsibilities (client, data storage, synchronization, authentication/authorization, and notifications/reminders) and will be designed to support testability and maintainability.
- Software Design: Design workflows that keep inventory maintenance low-friction (capture, update, lookup) and that remain understandable in shared households. Provide clear feedback when items are low, near-expiration, or shared, and present budget summaries in a way that supports planning rather than reactive purchasing.
- Software Process: The project will follow an Agile, feature-team structure. Each feature team owns its feature end-to-end (research, design collaboration, implementation, testing, and documentation). Work will be tracked and reviewed using a structured issue and pull-request workflow to keep changes traceable and reviewed. Continuous feedback from team walkthroughs and scenario reviews will guide refinement of requirements and UX decisions.

Span

The project's span focuses on typical student living environments (dorms, apartments, shared houses) where household management is informal and time-constrained. The initial span targets small households with a modest inventory and a small number of cohabitants, and it assumes frequent lightweight updates rather than occasional heavy maintenance.

- Specifics of the System: Users can maintain a centralized inventory organized by room and category, update item quantities, and optionally track expiration dates and priorities. Household members can choose how they want to be informed about low-stock and upcoming expirations, like low-stock and upcoming expirations. Shared households can view a common inventory and distinguish between shared and personal items to reduce duplicates and conflict. Budget tracking connects purchases to categories and provides basic spending summaries and reports.
- Target Audience and Expansion: The initial span targets students living independently, including those living with roommates. Expansion may include richer analytics (usage rates and trends), more advanced report types, optional integrations with external productivity or budgeting tools, and improved automation for restock planning. Any expansion will remain

centered on student needs and low-friction maintenance.

- Methodology and Maintenance: The project will proceed iteratively, refining documentation and implementation through validation and verification activities. Progress will be supported through iterative cycles of documentation, implementation, and validation/verification activities, refining scope and requirements as stakeholder feedback and early testing reveal gaps. The cycle of feedback and refinement will ensure the system remains practical for real student workflows and constraints.

1.3.2. Synopsis

This project addresses inefficiencies and stress in student household management that arise when tracking of essentials relies on memory or informal methods. The project will produce a system-to-be that supports lightweight tracking of household items and spending for students living alone or with roommates, with particular focus on minimizing maintenance effort, reducing duplicate purchases, and lowering the likelihood of items expiring unnoticed. Shared-household use will be supported so that cohabitants can coordinate around what is available and what needs replenishing.

Work will proceed through domain engineering and requirements engineering (including personas and scenarios), followed by architectural and design decisions and incremental implementation. Validation and verification activities (walkthroughs, scenario-based reviews, and testing) will be used to confirm that the evolving design matches stakeholder needs and that key behaviors work reliably. Across milestones, the team will iteratively refine the documentation and implementation based on evidence and feedback, aiming for an approach that remains practical for real student workflows over time.

1.4. Foundational Activities for System Development

A successful system-to-be is built on more than just code. The following activities establish the foundation upon which the Digital Home Inventory system will be developed, ensuring it accurately reflects student needs and performs reliably in real-world conditions.

1.4.1. Domain and Requirements Engineering

Learning how students actually live comes first. Through conversations and watching how people manage their daily routines—like using shared notes, talking things over with roommates, or simply trying to remember what they need—the project builds a picture of current habits that exist without any special application. Some students live alone, others share space with several people. Some organize carefully, others wing it. From these observations, the team will extract domain concepts (e.g., item types, locations, shared responsibility, replenishment routines, and expiration awareness) and translate them into stakeholder-centered needs, personas, and scenarios. These will then be refined into a first set of user stories and measurable requirements, with open questions explicitly recorded for later research. This reduces the risk of building requirements that do not match real student routines. To build this understanding, the team will use short interviews and guided walkthroughs of students' current practices, collect a small set of concrete "day-in-the-life" scenarios, and review feedback from existing inventory/list tools to identify recurring friction points.

1.4.2. Documentation and Knowledge Management

As the project grows, decisions about the application should not live only in people's heads or get buried in old messages. Writing things down in one place gives everyone access to the same information. Keeping documentation versioned and maintained alongside development work ensures it evolves with the system and remains easy to review and update. When someone joins later, they can read about why certain choices were made and how different pieces fit together. The same record helps keep everyone using the same words for the same things—whether "pantry" means only food or includes cleaning supplies, for example. Without this habit, people start remembering things differently or making assumptions that others do not share. The project becomes harder to keep consistent as new features get added over time.

1.4.3. Quality Assurance Planning

An application that surprises users or fails when they need it will not become part of anyone's routine. Knowing when a feature is truly finished means agreeing on what success looks like beforehand. In addition to scenario-based validation, the team will plan verification activities such as unit/integration tests for core behaviors and periodic walkthroughs to check consistency between terminology, requirements, and implementation. For alerts, success might mean a student finds out about expiring yogurt in time to eat it. For the shared list, it might mean roommates see the same information when deciding who goes to the store. Walking through made-up but realistic situations with actual students helps catch mismatches between what the team builds and what users actually need. Talking through examples together—like a week in the life of someone trying to stick to a grocery budget—shows whether the planned features will hold up in real life. The point is an application that students find genuinely helpful, not one that simply passes technical checks.

1.4.4. Project Management and Coordination

Several groups working on different concerns of the same system-to-be need to stay connected. Regular conversations between teams working on inventory, alerts, budgeting, and reports help spot potential trouble before it slows everyone down. A shared work-tracking board will make ownership and progress visible and help the team manage dependencies between features. Looking over each other's work before it becomes part of the main application catches small issues early. When the team working on reports needs information about spending, checking in with the budgeting team early saves redoing work later. These habits of talking and coordinating mean that when it comes time to bring everything together, the pieces fit without surprises.

1.4.5. Other Activities

This section tracks major updates and documentation changes since the Milestone 1 submission.

1.4.5.1. Milestone 2 Updates

Section	Description of Changes	New/Updated Files
2. Descriptive	Established a comprehensive visual identity and UI system including color theory and typography standards.	`color_palette.adoc`, `Typography-Style-Guide.adoc`, `UI_card_layout_strategy.adoc`

Section	Description of Changes	New/Updated Files
2. Descriptive	Defined technical implementation for automated background processes, including expiration checks and stock tracking logic.	`automated-stock-check-logic.adoc`, `automated-expiration-check-logic.adoc`, `barcode_scanning_manual_entry.adoc`
2. Descriptive	Finalized high-fidelity interactive screen designs for the three core modules: Inventory, Alerts, and Budget.	`core-inventory-wireframes.adoc`, `Alerts-&-Notifications-Screen.adoc`, `Budget-spending.adoc`
2. Descriptive	Detailed the software architecture for "Sweep" operations and created a robust notification deduplication strategy.	`design-and-document-sweep-architecture.adoc`, `notification-deduplication-strategy.adoc`, `sweep-failure-detection-recovery.adoc`
3. Analytic	Conducted concept analysis on inventory state changes, ownership permissions, and risk mitigation for automation.	`inventory_quantity_change_model.adoc`, `ownership_model_research.adoc`, `alerts-restock-automation-risk-analysis.adoc`
3. Analytic	Formulated state machines for the alert system and defined the decision logic for the grocery list auto-generator.	`alert-system-states.adoc`, `grocerylist_autoadd_decisionlogic.adoc`
3. Analytic	Developed data validation protocols and a comprehensive End-to-End (E2E) testing plan for the application.	`core-inventory-data-validation-rules.adoc`, `acceptance-e2e-tests.adoc`
4. Specific Topics	Expanded stakeholder analysis to include "non-obvious" users and drafted a plan for broad domain requirements acquisition.	`domain-requirements-acquisition.adoc`, `additional-stakeholders.adoc`, `stakeholder-perpective.adoc`
4. Specific Topics	Established inconsistency management standards for document versioning and household item hierarchies.	`inconsistency-management.adoc`, `consistency_hierachy_standards.adoc`, `document-duplicate-prevention-logic.adoc`

1.5. Derived Goals

1.5.1. Primary Derived Goals

In addition to the primary goal already established by the current situation and needs, this project pursues several secondary outcomes that guide decision-making across milestones. A key derived goal is to encourage more sustainable household routines among students by reducing avoidable waste over time—such as items expiring unused or being repurchased unnecessarily—through better day-to-day awareness of what is already on hand. Another derived goal is to improve adoption and continued use: the project should support habits that remain realistic under busy student schedules, so the practice of keeping track of essentials does not collapse after an initial

setup period.

A further derived goal is to strengthen coordination and trust in shared households. Even when people share expenses and supplies, misunderstandings about what is available and who will replenish items can create friction; the project aims to reduce these coordination breakdowns as a secondary social outcome of improved household awareness. Where budget awareness is in scope, an additional derived goal is to help students reflect on spending patterns around household essentials, supporting more informed planning rather than reactive purchases.

Finally, the project has internal quality goals that support sustainable progress across milestones. The team aims to maintain stable terminology and consistent documentation so decisions remain understandable and traceable over time, reducing rework and concept drift. The project will also emphasize evidence-based development by using validation and verification activities—such as personas, scenarios, walkthroughs, and testing—to continually check that the team's understanding matches stakeholder reality and that key behaviors work as intended.

1.5.2. Competing Goals and Resolutions

Although the project's derived goals reinforce one another in many ways, some of them also create natural tensions that must be handled explicitly. Identifying these tensions helps the team avoid inconsistent design decisions and produces derived requirements that clarify how the system should behave when important goals compete.

1.5.2.1. 1. Low-effort use vs. inventory accuracy and reliability

Competing goal A: Keep the system lightweight enough that students can realistically maintain it under busy schedules.
Competing goal B: Preserve inventory information with enough accuracy and completeness for reminders, shared coordination, and planning to remain trustworthy.

These goals compete because reducing friction often means reducing the amount of information required at entry time, but requiring too little information can make the inventory incomplete, stale, or unreliable. The project already emphasizes both low-effort maintenance and accurate inventory visibility, so the system must balance continued adoption with dependable data quality.

Resolution policy: The system should minimize required user effort during common workflows, but it must still collect and preserve the minimum information needed to support accurate inventory state and reliable system behavior.

Derived requirements:

- `DG-REQ-01` Users shall be able to create an initial item record with only the minimum information needed to identify and track that item, while adding richer metadata later as needed.
- `DG-REQ-02` Low-stock and expiration-related behaviors shall only occur when the item information required to evaluate those conditions is present and valid.
- `DG-REQ-03` Recorded item metadata shall remain stable and trustworthy over time unless an authorized user explicitly changes or removes it.

- `DG-REQ-04` Routine item updates shall remain lightweight enough to fit ordinary student workflows without creating excessive maintenance burden.

Trace hooks: `N1`, `N2`, `N3`; derived goal for adoption and continued use; reliability quality need; `US-INV-01`, `US-INV-02`.

1.5.2.2. 2. Timely reminders vs. avoiding notification fatigue

Competing goal A: Ensure users are reminded in time about low-stock and expiration conditions.
Competing goal B: Avoid excessive or poorly timed alerts that cause frustration, desensitization, or notification disablement.

These goals compete because a highly proactive reminder strategy may reduce the chance that important conditions are missed, but it also risks overwhelming users and undermining trust in the system. The project goals explicitly include automated low-stock reminders, expiration alerts, and customizable thresholds, while also emphasizing that notifications should remain helpful and relevant.

Resolution policy: The system should favor timely reminders for important conditions, but it must rate-limit and deduplicate recurring alerts so persistent states remain visible without becoming spam.

Derived requirements:

- `DG-REQ-05` Users shall receive low-stock reminders often enough to support timely replenishment, but not more than once per item for the same household member group within a single calendar day.
- `DG-REQ-06` Users shall receive expiration-related reminders often enough to support timely action, but not more than once per item for the same household member group within a single calendar day unless a new alert condition is created by a state change.
- `DG-REQ-07` Repeated sweep executions shall not produce duplicate notifications for the same item-condition pair within the same daily reminder period.
- `DG-REQ-08` Reminder behavior shall favor relevance and timeliness over notification volume by limiting prompts to conditions supported by valid item data and active alert rules.

Trace hooks: `N2`; usability and reliability quality needs; `Lightweight Smart Inventory`; `US-ALT-01`.

1.5.2.3. 3. Shared household visibility vs. privacy and least privilege

Competing goal A: Increase household coordination by making shared inventory visible to cohabitants.
Competing goal B: Protect user-specific or sensitive information by exposing only the information necessary for coordination.

These goals compete because coordination improves when household members can see what is available and what needs replenishment, but excessive visibility can expose ownership, spending, or personal-item information that not everyone should access. The project scope includes both shared inventory coordination and privacy/security needs, so the system must support collaboration without ignoring access boundaries.

Resolution policy: The system should expose the minimum shared information required for household coordination while restricting user-specific metadata to the appropriate scope and permissions.

Derived requirements:

- `DG-REQ-09` Household members shall be able to see the availability and replenishment-relevant status of shared items.
- `DG-REQ-10` Shared household data and personal or user-specific item data shall remain distinguishable when both exist in the same household context.
- `DG-REQ-11` Ownership, spending, and other user-specific item information shall only be visible to users with permission to view it.
- `DG-REQ-12` The distinction between shared items and personal items shall remain clear whenever that distinction affects replenishment or household coordination.

Trace hooks: `N4`, `N5`; derived goal for shared-household coordination and trust; security/privacy quality need; `US-SB-01`.

1.5.2.4. Summary of trade-off handling

These competing-goal resolutions clarify that the system must not optimize convenience, reminders, or transparency in isolation. Instead, the project balances adoption, data trustworthiness, alert usefulness, and household coordination by introducing derived requirements that are concrete enough to guide implementation and later validation and verification.

2. Descriptive Part

2.1. Domain Description

2.1.1. Domain Rough Sketch

Raw, lightly organized domain notes and examples. The goal of this section is to preserve concrete observations and participant wording as directly as possible before later interpretation.

Most quotes were translated from Spanish. Real names are omitted and replaced with stable participant labels so that statements from the same participant can still be traced consistently across the document.

2.1.1.1. Interview Snippets and Raw Observations

Shared Apartment - 3 Students (Interview, Jan. 20, 2026)

- **Student 1**: "If it is not on our fridge note, we assume we are out."
- **Student 2**: "Sometimes I check the pantry before leaving, but by the time I get to Walmart I forgot what we needed."
- **Student 3**: "We have rice in two places. One bag in the kitchen cabinet and another in a plastic bin in the hallway."
- Current tracking method observed: fridge sticky note + WhatsApp group chat.
- Observed responsibility pattern: no one described being formally responsible for keeping track of quantities.
- Shared example reported: milk is sometimes bought twice in the same week.
- When asked how they know something is low, participants answered with phrases such as "when it feels light" and "when we open the backup."
- The backup opening is not usually written down.

Student 4 - Living Alone (Interview, Feb. 2, 2026)

- **Student 4**: "I buy in bulk from Costco when I go with my parents."
- **Student 4**: "I got like 24 yogurts once."
- **Student 4**: "Some of them expired."
- Follow-up note: participant estimated throwing away several yogurts during the last semester.
- **Student 4**: "I keep cleaning supplies in the bathroom closet and under the kitchen sink."
- **Student 4**: "I don’t count exactly. I just look."
- **Student 4**: "I use the Notes app sometimes, but then I stop."
- **Student 4**: "If it takes more than a few seconds to log, I won’t do it."

Shared Household - 2 Adults, 1 Child

- One household member reported inviting family members to a shared grocery list app.
- One adult reportedly does most purchases.
- Reported situation: a child may use the last unit of an item without telling the others.
- Reported example: toilet paper runs out unexpectedly when guests visit.
- Medicines are kept in a separate drawer because expiration matters more there.
- Reported situation: expired medicine was discarded without realizing there was another unopened box.
- Reported expectation: access should be removable when someone moves out.

Student 5 - Student Athlete (Interview, Feb. 2, 2026)

- **Student 5**: "During exam weeks I eat way more snacks."
- **Student 5**: "Protein bars last me like two weeks normally, but during midterms maybe four or five days."
- **Student 5**: "I don't really adjust it formally. I just buy more when they're gone."
- **Student 5**: "I just remember how fast it ran out last time."

Observations from Informal Brainstorming Session

- Participants often describe availability with words like "full", "almost empty", or "backup", and less often with exact numbers.
- Items repeatedly mentioned as essentials included toilet paper, soap, toothpaste, milk, and rice.
- Perishables repeatedly mentioned included yogurt, vegetables, chicken, and medicine.
- Bulk buying was usually justified by lower price per unit.
- Waste was mentioned more often for food than for cleaning supplies.
- Duplicate buying was mentioned most often for milk, eggs, and rice.
- Some participants organize by location such as kitchen, bathroom, or laundry area; others do not.
- No participant described doing a periodic full inventory check.
- Multiple participants said they have forgotten what they already had at home.
- Some participants said they prefer taking a pantry picture instead of writing a list.

Edge Situations Mentioned

- Barcode does not scan because packaging is damaged.
- Expiration date is unreadable or missing.
- Guests consume items faster than usual.
- Someone moves out and still has access to the shared list.
- Backup is stored in a car trunk and forgotten.

NOTE	A previously stated edge case claimed that items bought in bulk may have multiple different expiration dates in the same package. Because that statement is not yet supported by a clearly attributable source in the current rough sketch, it is not treated as established domain evidence here.

2.1.2. Terminology

This section consolidates the domain's ubiquitous language into a single annotated reference. Every term is tagged with the phase in which it was first introduced:

- **Domain** — emerged from domain analysis (interviews, observations, informal practices)
- **Requirements** — introduced when specifying capabilities of the system-to-be

Term	Type	Phase	Definition	Synonyms / Notes
Household	Entity	Domain	A living unit (dorm, apartment, shared house) where one or more people manage household essentials.	Previously "Home."
Household member	Entity	Domain	A person who belongs to a household and participates in consuming, replenishing, or coordinating around items.	Previously "Owner" or "Occupant."
Invitation	Entity / Event	Domain	A temporary request that allows a participant to join a shared household context.	Important states may include sent, accepted, declined, expired, or revoked.
Membership	Relationship / Concept	Domain	The relationship connecting a participant to a household for shared coordination and participation.	Changes over time through joining, leaving, removal, or invitation acceptance.
Location	Entity	Domain	A storage place within a household (kitchen cabinet, pantry area, bathroom closet, car trunk, hallway bin).	Previously "Room."
Item type	Entity	Domain	The kind of an item, independent of a specific package (e.g., "Milk", "Dish soap", "Toilet paper").	Previously "Product."
Item instance (package)	Entity	Domain	A concrete package or unit of an item type that physically exists in the household (e.g., a specific milk carton).	Previously "Stock."
Inventory (reality)	Entity	Domain	The set of item instances that physically exist in the household at a given time.	Real-world state, not a list or record.

Term	Type	Phase	Definition	Synonyms / Notes
Spending category	Entity / Label	Domain	A grouping for spending (e.g., groceries, cleaning, personal care). May align with item categories, but is not required to.	—
Budget	Entity / Rule	Domain	A financial limit defined for a period (household-level or category-level) used to evaluate spending.	Budget window and cycle can vary.
Tracked inventory record	Artifact	Domain	A representation of availability used by household members (fridge note, list, memory, chat message). May diverge from reality.	Supports the "tracked vs. reality" distinction.
Restock list / planner	Artifact	Domain	A household-oriented list of items that should be replenished soon (fridge note, notes-app list, chat).	Delivery method is not part of the domain.
Alert / notification (domain-level)	Artifact / Event	Domain	A message that informs a household member about an actionable condition (low-stock, out-of-stock, expiring, expired, budget threshold crossed).	Represents that an actionable condition has been communicated.
Spending entry	Record	Domain	A record of a financial transaction in a household (purchase, refund, or adjustment), optionally associated with an item type and spending category.	Supports budgeting and reporting.
Transaction (quantity change)	Record	Domain	A record of an availability change (consume, restock, or adjust) with a time and reason.	Supports traceability and verification.
Availability	Property	Domain	A household's current understanding of how much of an item is left, often qualitative ("full", "almost empty", "backup").	May later be represented as quantity.
Quantity	Property	Domain	A more precise measure of availability (count or units). Can be unknown or approximate; must never be negative.	—
Expiration date	Property	Domain	A date after which an item instance is considered expired or undesirable. Applies to some items, not all; may be unreadable or missing.	—

Term	Type	Phase	Definition	Synonyms / Notes
Low-stock threshold	Rule / Property	Domain	A boundary value used to decide when an item is considered low.	May be a fixed quantity initially.
Budget cycle (period)	Concept	Domain	The time window over which spending is compared to a budget (weekly, monthly, custom). Resets and rollovers are policy decisions.	—
Backup supply	Practice / Concept	Domain	A household practice of keeping a spare instance or package to reduce the risk of running out.	Often used instead of numeric tracking.
Consumption	Action	Domain	Using an item over time, reducing its availability. Rate may vary by context (exam weeks, guests).	—
Replenishment (restock)	Action	Domain	Acquiring more of an item, increasing availability. Often triggered after low-stock, out-of-stock, or expiration events.	—
Snooze / defer	Action	Domain	A household member delays acting on an alert for a period of time.	The household member may receive another alert for the same actionable condition later.
Dismiss / ignore	Action	Domain	A household member acknowledges an alert and chooses not to act on it.	—
Low-stock judgment	Event / Rule	Domain	A decision that an item is running low, based on a household strategy ("feels light", "opened backup") or a numeric threshold.	Not inherently numeric.
Out-of-stock	State / Event	Domain	The condition or event where an item is considered unavailable because it is gone (quantity at zero, or judged empty).	Distinct from low-stock.
Near-expiry	State / Event	Domain	An item instance approaching its expiration date within a defined window (e.g., N days).	Window length can vary.
Expired	State / Event	Domain	A item instance past its expiration date.	Expired does not always mean unavailable; policy varies by household.
Budget threshold crossing	Event	Domain	The moment when spending reaches or exceeds a defined threshold (e.g., 80%, 100% of the budget).	Can trigger alerts, subject to anti-spam rules and the budget cycle.

Term	Type	Phase	Definition	Synonyms / Notes
Item acquired	Event	Domain	A household member obtains an item instance (possibly in bulk) and brings it into the household.	"Bought 24 yogurts from Costco."
Item stored	Event	Domain	An item instance is placed in a household location.	"Rice stored in kitchen cabinet; backup in hallway bin."
Item consumed	Event	Domain	A household member uses an item, decreasing its availability.	Consumption rate may increase during exams or when guests visit.
Backup opened	Event	Domain	A household member begins using the backup supply of an item.	Often used as a qualitative low-stock signal.
Replenishment decided	Event	Domain	A household member decides the household should acquire more of an item type.	May be prompted by low-stock, out-of-stock, or expiration.
Item replenished	Event	Domain	New item instances of an item type are acquired to restore availability.	Can be duplicated if coordination fails.
Duplicate purchase	Event	Domain	More of an item type is acquired unnecessarily because household members acted on inconsistent information.	Common for milk, eggs, and rice per interviews.
Item approaching expiration	Event	Domain	An item instance is within a near-expiry window.	Motivates timely use or discard decisions.
Item expired	Event	Domain	An item instance passes its expiration date (if it has one).	Policy on what to do varies by item type and household.
Item discarded	Event	Domain	A household member throws away an item instance (expired, spoiled, or unwanted).	Can occur even when another unopened instance exists.
Alert generated	Event	Domain	A low-stock, out-of-stock, near-expiry, expired, or budget condition becomes actionable and is communicated to household members.	—
Alert dismissed / deferred	Event	Domain	A household member acknowledges an alert and chooses to ignore it or act later.	Supports snooze and anti-spam behavior.
Restock entry created / updated	Event	Domain	An item is added to or updated on the restock list due to low-stock, out-of-stock, or expired conditions.	May be manual or derived from judgments.

Term	Type	Phase	Definition	Synonyms / Notes
Spending recorded	Event	Domain	A household member logs a spending transaction (purchase, refund, or adjustment).	May be shared or personal.
Budget cycle started / ended	Event	Domain	A new budget period begins or a period closes.	Supports reset and rollover policies.
Household membership changed	Event	Domain	A member moves in or out; household participation changes.	Motivates access removal expectations later.
User	Actor	Requirements	A person who interacts with the system-to-be through its interface. Corresponds to a household member in the domain.	# "User" is the system-facing term; "Household member" is the domain-facing term.#
Household admin	Role	Requirements	The single household member with administrative responsibility at a given time. Transferable before the admin leaves.	Domain equivalent: informal "household manager" role.
Membership state	Property	Requirements	The status of a user's participation in a household (active, pending, revoked).	No direct domain artifact.
Reorder point (threshold)	Rule / Property	Requirements	A quantity value at or below which an item type is considered low. Initial requirements may accept a fixed per-item threshold.	Refines the domain's "low-stock threshold" with a numeric constraint.
Near-expiry window	Rule / Property	Requirements	The definition of "expiring soon" — the number of days before expiration that triggers a near-expiry alert.	Window value is a requirements decision deferred from the domain.
Deduplication / anti-spam rule	Rule	Requirements	A constraint preventing repeated alerts or repeated restock-list additions when no meaningful state change has occurred.	Operationalizes the domain's "should not cause repeated alerts."
Actionable state	Concept	Requirements	A system-recognized condition (low-stock, out-of-stock, near-expiry, expired, budget-threshold-crossed) that may trigger an alert or restock entry.	Refines domain-level events into system-detectable states.
Budget status (derived)	Concept	Requirements	A computed view of spending versus budget within a cycle: amount spent, amount remaining, percentage used, and threshold flags.	Derived from spending entries; not directly observable in the domain.

Term	Type	Phase	Definition	Synonyms / Notes
Scheduled sweep	Function	Requirements	A periodic background check that evaluates low-stock and expiration conditions without requiring the user to open the system.	Automation mechanism; delivery details are implementation-level.
Notification record	Record	Requirements	A persisted representation of an alert, including type, timestamp, and read/unread status, available at minimum for in-app review.	Refines the domain-level "alert"; adds persistence and status tracking.

2.1.3. Narrative

A cohesive story of how the domain works, independent of any specific software system.

2.1.3.1. Overview

Students living independently go through a continuous cycle of acquiring, using, and replacing household items. They buy food and supplies when they have time and money, store them in locations within their living space, and consume them gradually. When items run low or run out, someone must notice and take action to replenish them. In shared households, this cycle involves multiple household members who must coordinate—or fail to coordinate—about what is needed, who will buy it, and when.

Tracking is typically informal. Many households rely on memory, a fridge note, a chat message, or a quick pantry check. This tracked understanding can diverge from reality: items may exist but be forgotten in a different location, backups may be opened without being recorded, and purchases may be duplicated because household members act on incomplete information. Waste can occur when items expire unnoticed or when duplicates are purchased while another instance is already available.

Consumption patterns are not stable. During exam weeks or when guests visit, items can be used faster than usual. Bulk purchases can increase the number of item instances in the household at once, sometimes with different expiration dates within a single bulk pack. The household cycle repeats constantly under constraints of limited time, attention, and coordination.

Some households also keep an informal understanding of spending on essentials. Purchases may be remembered loosely, estimated from receipts, or discussed among roommates when money feels tight. When spending is tracked (even imperfectly), it influences decisions such as buying in bulk for lower unit price, delaying non-urgent restocks, or choosing cheaper alternatives. In shared households, disagreements can arise when it is unclear which purchases are personal versus shared or when one member repeatedly pays for shared items. Budget awareness therefore becomes another dimension of household coordination, alongside item availability and replenishment.

2.1.3.2. Actors and artifacts

Actors (domain roles):

- Students living alone (single-member household)
- Students living with roommates/cohabitants (multi-member household)
- Household members who often coordinate replenishment (informal “household manager” role)
- Household members who focus on budget awareness (budget-conscious role)

Artifacts used today (domain-visible):

- Fridge notes / whiteboards
- Chat messages (e.g., group chat)
- Notes app lists
- Pantry photos
- Receipts or memory-based estimates

2.1.3.3. Typical domain flow (as observed)

1. Household members acquire items (sometimes in bulk) and store them in household locations.
2. Items are consumed over time; availability decreases.
3. A household member checks availability using informal artifacts (note, memory, pantry check).
4. If the member judges an item to be low or out (often using qualitative strategies like “feels light” or “opened backup”), replenishment is triggered.
5. In shared households, coordination may succeed (one person buys) or fail (duplicate buying).
6. Some items expire before use; households may discover this late and discard them.
7. Household members may keep informal awareness of spending; when tracked, spending influences replenishment choices and can create shared-vs-personal disputes.
8. Membership changes (move-out) require the household to update who participates and who is informed.

2.1.4. Entities

Key domain entities and artifacts. The entities below follow the terminology established in Terminology and use a single unified concept vocabulary for the domain.

Entity	Description	Key attributes
Household	A living unit where household essentials and spending are managed.	name (optional label), members, locations, inventory (reality), optional budgets
Household member	A person who belongs to a household and consumes, replenishes, or coordinates around household items.	name/identifier (domain-level), household reference, informal role

Entity	Description	Key attributes
Location	A storage place within a household where items are kept.	name/label, optional purpose
Item type	A definition of an item independent of a specific package or purchase.	name, optional category, optional brand/size/variant, optional barcode
Item instance (package)	A concrete package or unit of an item type that exists in the household.	item type reference, location reference, ownership (personal/shared), quantity/availability, optional purchase date, optional expiration date, optional notes
Inventory (reality)	The set of item instances that physically exist in the household at a given time.	household reference, item instances
Tracked inventory record	A representation households use to reason about availability. It may diverge from reality.	artifact type (note/chat/list/memory), item reference (explicit or implicit), last-updated (if recorded)
Restock list / planner	A list of items judged to need replenishment soon.	entries (item types), reason (low/out/expired), optional responsibility marker, status (active/deferred)
Alert / notification record	A record of an alert-worthy condition presented to household members.	type (low/out/near-expiry/expired/budget), target (item type/instance or budget), created time, read/unread, dismissed/deferred
Spending entry	A logged financial transaction event within a household.	amount, date/time, payer (household member), scope (shared/personal), optional item type reference, category, notes, transaction type (purchase/refund/adjustment)
Spending category	A grouping label used to organize spending by type or purpose.	name, optional description
Budget	A spending limit defined for a period.	scope (household/category), limit amount, cycle (period), thresholds (warning levels), visibility rules
Budget status (derived)	A derived view of spending versus budget within a cycle.	spent-to-date, remaining, percent used, threshold-crossed flags
Invitation	A temporary request that allows a new participant to join a shared household context.	sender, recipient, target household, status, expiration timestamp

Entity	Description	Key attributes
Membership	The relationship connecting a system participant to a household for coordination and access purposes.	household reference, participant reference, role, membership state

NOTE

Earlier drafts used terms such as **Home**, **Room**, **Product**, **Stock**, **Owner**, and **Member**. In this document, those terms are replaced by the unified vocabulary defined in the terminology section:

- Home → Household
- Room → Location
- Product → Item type
- Stock → Item instance (package)
- Owner / Member → Household member, with more specific role language used only where needed

2.1.5. Events

Domain events represent instantaneous state changes in the domain that matter to stakeholders. Each event is phrased in the past tense to emphasize that a state change has just occurred. Events are independent of any particular system-to-be, even if a later system represents or reacts to them.

Event	Description	Notes / examples
Item instance acquired	A household member has just obtained an item instance and it has become available in the household.	”24 yogurts have just arrived from Costco.” Two related state changes: the obtaining and the becoming-available-in-household may be distinct events in practice.
Item instance stored	An item instance has just been placed in a household location.	”Rice has just been stored in kitchen cabinet; backup placed in hallway bin.”
Item instance consumed	A household member has just used an item, and its availability has decreased.	Consumption rate may change during exams/guests.
Backup opened	A household member has just begun using the backup supply of an item.	Often used as a qualitative low-stock signal.
Low-stock judged	An item has just been judged as running low.	”Feels light” / “backup opened” / threshold crossed.
Out-of-stock reached	An item’s availability has just reached zero.	”Toilet paper has just run out unexpectedly.”

Event	Description	Notes / examples
Replenishment decided	A household member has just decided the household should acquire more of an item type.	May be prompted by low-stock, out-of-stock, or expiration.
Item instance replenished	New item instance of an item type have just been acquired, restoring availability.	Can be duplicated if coordination fails.
Duplicate purchase occurred	More of an item type has just been acquired unnecessarily because members acted on inconsistent information.	Common for milk/eggs/rice per rough sketch.
Item instance reached near-expiry	An item instance has just entered its near-expiry window.	Motivates timely use or discard decisions.
Item instance expired	An item instance has just passed its expiration date.	Policy varies by item type/household.
Item instance discarded	A household member has just thrown away an item instance.	Can occur even when another unopened instance exists.
Alert generated	A low/out/near-expiry/expired/budget condition has just become actionable and has been communicated to household members.	Delivery method is not part of the domain event.
Alert dismissed / deferred	A household member has just acknowledged an alert and chosen to ignore it or act later.	Supports “snooze” and anti-spam behavior.
Restock entry created/updated	An item has just been added to the restock list/planner due to low/out/expired conditions.	May be manual or derived from judgments.
Spending recorded	A household member has just logged a spending transaction.	May be shared or personal.
Budget threshold crossed	Spending in a budget cycle has just reached a warning level or exceeded the limit.	May trigger an alert.
Budget cycle started/ended	A new budget period has just begun, or a period has just closed.	Supports reset/rollover policies.
Household membership changed	A member has just moved in or out; household participation has changed.	Motivates access removal expectations later.

2.1.6. Functions

Major domain functions/capabilities, described independently of UI. Each function includes a signature that names its inputs and outputs so that the effect of the operation can be stated precisely.

2.1.6.1. **Stock item instance**

Introduce new item instance of an item type into a household location, whether as a first stocking action or as replenishment. After stocking, checking the location will show that the given quantity of the item type has been added there.

```
#stockItem : Household x ItemType x Location x QuantityDelta x ExpirationDate? → Household#
```

- *Location* determines where the new item instances are placed.
- *QuantityDelta* is the number of units added.
- *ExpirationDate* is optional; if provided, it is associated with the new item instances.
- The returned Household reflects the updated inventory.

2.1.6.2. **Check location**

Inspect a location to determine which item types it contains and in what quantities. This is an observation function.

```
#checkLocation : Household x Location → List (ItemType x Quantity)#
```

- Returns all item types stored at the given location along with their current counts.
- This function enables expressing pre/post-conditions of other functions. For example, after calling `stockItem` with 3 units of rice at the kitchen cabinet, `checkLocation` for the kitchen cabinet will show the rice count increased by 3.

2.1.6.3. Move item instance

Change where an item instance is stored. After moving, checking the old location will no longer show this instance, and checking the new location will include it.

```
moveItemInstance : Household x ItemInstance x Location → Household
```

- *Location* is the destination where the item instance will be placed.

2.1.6.4. **Record consumption**

Decrease availability of an item instance due to use.

```
#consume : Household x ItemInstance x QuantityDelta → Household#
```

- *QuantityDelta* is the number of units consumed.
- After consumption, the item instance's quantity is reduced by the delta. If the new quantity reaches zero, the item is considered out-of-stock. If the new quantity falls at or below the low-stock threshold, the item is considered low-stock.

2.1.6.5. Judge low-stock

Determine whether an item type is considered low based on household strategy or threshold.

```
#isLowStock : ItemType x Availability x Threshold? → Bool#
```

- If a *Threshold* is provided and *Availability* is at or below it, returns true.
- If no threshold is set, the function may rely on qualitative strategies.

2.1.6.6. Determine expiration status

Classify an item instance's expiration state when an expiration date exists.

```
#expirationStatus : ItemInstance x DateNow → (Active | NearExpiry | Expired |
Unknown)#
```

- Returns *Unknown* when the item instance has no expiration date or the date is unreadable.
- *NearExpiry* applies when the item is within the near-expiry window.

2.1.6.7. Create restock entry

Add an item type to the restock list/planner with a reason.

```
#addToRestockList : RestockList x ItemType x Reason → RestockList#
```

- *Reason* indicates why restocking is needed (LOW_STOCK, OUT_OF_STOCK, EXPIRED).

2.1.6.8. Defer / dismiss restock entry

Allow a household member to postpone or remove an entry from restock planning.

```
#deferRestock : RestockList x ItemType x Until? → RestockList#
```

- *Until* is an optional date/time for when the entry should reappear.

2.1.6.9. Generate alert

Create an alert record when an actionable state is reache.

```
#generateAlert : AlertLog x AlertType x Target → AlertLog#
```

- *AlertType* is one of: LOW_STOCK, OUT_OF_STOCK, NEAR_EXPIRY, EXPIRED, BUDGET_THRESHOLD.
- *Target* is the item type, item instance, or budget that triggered the condition.

2.1.6.10. Acknowledge alert

Record a household member's decision to dismiss or defer an alert.

```
#ackAlert : AlertLog x Alert x Action(dismiss|defer) x Until? → AlertLog#
```

2.1.6.11. Record quantity change transaction

Append a transaction describing why and when availability changed.

```
#recordTransaction : TransactionLog x ItemInstance x Change x Reason x Time →
TransactionLog#
```

2.1.6.12. Record spending event

Create a spending entry capturing who paid, how much, when, and whether it is shared or personal.

```
#recordSpending : Household x Amount x Date x Payer x Scope x ItemType? x Category? →
Household#
```

- *Payer* is the household member who made the purchase; attribution is retained even after inventory corrections.
- *Scope* determines whether the spending is shared among household members or personal.

2.1.6.13. Adjust spending entry

Correct spending records via adjustment/refund entries rather than destructive edits.

```
#adjustSpending : Household x SpendingEntry x Adjustment → Household#
```

2.1.6.14. Compute budget status

Compute spent-to-date and remaining budget for a household or category within a budget cycle.

```
#budgetStatus : Budget x List SpendingEntry x DateNow → BudgetStatus#
```

2.1.6.15. Detect budget threshold crossing

Determine whether spending crosses warning/limit thresholds.

```
#budgetThresholdCrossed : BudgetStatus x Thresholds → Bool#
```

2.1.6.16. Domain Logic for Spending Tracking

Spending tracking in this domain represents how household purchases are recorded, attributed, and preserved over time. A spending event exists only when a financial transaction occurs within a household context, and it supports later budgeting, reporting, and accountability without depending on any specific implementation.

A spending event captures: * who paid * what was purchased * the monetary amount * when the transaction occurred * whether the cost is personal or shared

Core rules

- Spending is recorded only when a purchase with a defined cost exists. Creating or editing non-financial item information does not by itself create spending.
- Spending is always associated with a household context. It does not exist independently of the shared environment in which the purchase occurred.
- Every spending event must identify a payer. Anonymous financial records are not allowed.
- Spending must distinguish between personal and shared attribution. Shared attribution does not remove payer identification.
- Spending records are historical and must remain traceable. Deleting or changing inventory items does not erase past spending.
- Spending depends on a valid purchased item or recorded purchase context. Orphaned financial records are not valid.
- If cost information is corrected, the correction must preserve traceability through a new entry or other explicit record of change rather than silently overwriting history.

Implications

These rules ensure that spending remains tied to household activity, preserves accountability, and can later support budgeting, reporting, and reimbursement logic without redefining the core domain concepts.

2.1.7. Behaviors

Recurring behaviors or rules in the domain: patterns, policies, and "how things are done."

2.1.7.1. Behavior patterns

- **Restock cycle:** A household member consumes an item instance → availability decreases → low-stock is judged or quantity crosses a threshold → a replenishment decision is made → more of the item type is acquired.
- **Shared purchase flow:** A household member buys a shared item → the purchase is recorded → the shared household understands that the cost belongs to the shared context rather than only one person.
- **Expiration monitoring:** An item instance approaches its expiration date → the household notices or is informed → the item is used, discarded, or left in place depending on household practice.

2.1.7.2. Domain rules

- One barcode corresponds to exactly one **Item type** definition.
- A duplicate **Item instance** entry submitted within a short window may be treated as an addition to an existing tracked record rather than as a separate new record, unless household members explicitly keep them separate.
- **Spending entries** are historical records; inventory corrections should not silently overwrite financial history.
- A **Household** has one administrative responsibility at a time when that role is formally represented.
- If that administrative responsibility is transferred, the new responsibility must be established before the previous admin leaves the household context.

2.1.7.3. Edge cases

- If an item instance has no expiration date, expiration-based behavior cannot be derived from date information alone.
- If a **household admin** attempts to leave without transferring that responsibility, shared-household administration becomes unresolved until another member takes over.

2.1.7.4. Shared-Household Invitation and Membership Changes

In shared households, participation may change over time as new members are invited, current members leave, or access is revoked. Invitation is therefore part of how household membership changes, even though the exact delivery and security mechanism belong to later requirements and implementation sections.

A household invitation is a temporary request that allows a participant to join a household context. At the domain level, the important states are:

- sent
- accepted
- declined
- expired

- revoked

These states matter because they affect who participates in household coordination and who should continue to receive shared information.

2.1.8. Domain Invariants

This section states a small set of stable domain rules that must remain true across the household inventory domain.

2.1.8.1. Core invariants

- A household has one or more household members while it is active.
- An item instance belongs to exactly one household context.
- An item instance is stored in some location, even if that location is forgotten or described informally.
- A household may contain multiple item instances of the same item type at the same time.
- A household member's participation in a household may change over time.
- Shared-household coordination depends on knowing which items are shared, which are personal, and who is allowed to view or manage them.
- If spending is recorded, it remains associated with the household context in which it occurred.
- If an item instance has an expiration date, it may become expired without being consumed.

2.1.8.2. Ownership and participation notes

Ownership and responsibility may vary by household practice. Some item instances are treated as personal, while others are treated as shared. For that reason, the domain must support both personal and shared responsibility without redefining the core concepts of household, household member, item type, and item instance.

Rather than introducing a separate set of competing ownership entities, the document keeps the shared-household context in the concepts of household membership, personal/shared item status, and household coordination.

2.2. Requirements

This section captures initial requirements at multiple levels (stories, features, epics, constraints).

2.2.1. User Stories

The following user stories describe the core user-visible behaviors of the Digital Home Inventory system for Milestone 1. Each story is written from the perspective of a persona and is supported by measurable acceptance criteria. The story identifiers use prefixes: US-INV (Core Inventory), US-ALT (Alerts & Restock), US-SB (Sharing & Budget), and US-RPT (Reports & Insights).

2.2.1.1. US-INV-01: Add items to inventory

As **Mateo** (busy solo student), I want to **quickly add items to my tracked inventory after shopping**, so that **I can reliably know what I have without relying on memory**.

Also affects: Sofia, Julian, Andrea **Priority:** High

Acceptance Criteria * The system provides a prominent "Add Item" entry point from the inventory view. * Adding an item requires only minimal fields (name + quantity/availability), with optional fields (category, location, ownership status, expiration date). * Creating an item takes low effort (goal: can be completed in under 1 minute for a small set of items, assuming typical mobile input). * After saving, the item appears in the inventory list without requiring a manual refresh. * The user can edit an item from the inventory list (e.g., correct name/variant), and the updated information is reflected consistently wherever it appears.

> **NOTE** Expiration dates are optional. If an expiration date is unknown or unreadable, the system treats expiration status as unknown and avoids false "expired" alerts.

2.2.1.2. US-INV-02: See what belongs to whom

As **Sofia** (budget-conscious roommate), I want to **see which items are shared and which belong to specific people**, so that **I can avoid conflict and protect my personal purchases**.

Also affects: Julian, Andrea **Priority:** High

Acceptance Criteria * Every item in a shared household includes an ownership indicator: personal (linked to a household member) or shared. * The item detail view shows ownership status and relevant metadata (purchase date/cost when recorded). * When adding/editing an item, the user can set ownership to personal or shared. * Changing ownership from personal shared updates the tracked record and becomes visible consistently to household members after synchronization. * Spending views (if enabled) separate personal spending from shared spending so users can understand personal vs household costs.

2.2.1.3. US-ALT-01: Receive low-stock warnings

As **Julian** (diet-constrained student), I want to **receive low-stock warnings for items I rely on**, so that **I can reorder in time and avoid unexpected shortages**.

Also affects: Mateo, Andrea **Priority:** High

Acceptance Criteria * The user can set a low-stock threshold for an item (with sensible defaults if no threshold is set). * When an item's tracked quantity crosses from NORMAL/IN_STOCK to LOW_STOCK (threshold boundary), the system flags the item as low-stock. * Low-stock alerts are generated primarily on state transitions (e.g., NORMAL → LOW_STOCK) and are deduplicated (re-running checks without state change does not spam alerts). * When multiple items are low, the system may group notifications or present a summarized view to avoid overwhelming users. * Low-stock status remains consistent across views (inventory list, restock list/planner, dashboard summaries where applicable).

2.2.1.4. US-SB-01: Split shared purchases with roommates

As **Andrea** (unofficial household manager), I want to **record shared purchases and split costs with roommates**, so that **reimbursements are clear and disputes are reduced**.

Also affects: Sofia, Mateo, Julian **Priority:** High

Acceptance Criteria * The user can record a purchase with total cost and date, and mark line items (or the purchase) as shared vs personal. * The system can compute a split for shared items among selected household members (default equal split; adjustable when needed). * Household members can view the breakdown of a shared purchase (what was bought, how the split was calculated). * Payments can be marked as paid/unpaid, and the system maintains a simple reimbursement status view. * Disputes can be flagged on a purchase line item (recording that a discussion is needed) without deleting transaction history.

2.2.1.5. US-RPT-01: See household dashboard with insights

As **Andrea** (unofficial household manager), I want to **open one screen and quickly see what needs attention**, so that **I can plan restocking without guessing or repeatedly messaging roommates**.

Also affects: Sofia, Mateo, Julian **Priority:** Medium

Acceptance Criteria * The dashboard presents a summary view including: number of low-stock items, number of expiring/expired items (when expiration dates exist), and current-period spending (if spending is enabled). * The user can filter the dashboard by location (e.g., kitchen/bathroom) and return to the unfiltered view. * Summary counts update after changes are recorded and synchronized (avoid stale information). * Low/expiring summaries include location context when available (e.g., “toilet paper low in bathroom”). * The dashboard does not require users to navigate multiple screens to find the “top-level” status of the household.

2.2.1.6. Features

Features describe user-visible capabilities of the system-to-be at a level above individual requirements. The list below is intentionally concise for Milestone 1 and is grounded in the alert/grocery-list artifacts (auto-check output, trigger priorities, and the grocery list user flow).

ID	Feature	Primary value	Related stories / notes
F-INV	Inventory tracking (create/update items)	Enables a reliable tracked view of household items (quantity, location, ownership, optional expiration).	Supports US-INV-* (core inventory stories). Basis for alerts and grocery list generation.

ID	Feature	Primary value	Related stories / notes
F-ALERT	Alert generation from actionable state changes	Notifies household members when tracked conditions become actionable (e.g., LOW_STOCK, OUT_OF_STOCK, NEAR_EXPIRY/EXPIRED), with deduplication based on state transitions.	Supports US-ALT-* and the “alert trigger logic” notes. Delivery method can be in-app initially; push may be later.
F-AUTOCHECK	Automated daily check (AutoCheck) over inventory	Produces an “AutoCheck Output” (items + status results) used to drive alerts and grocery list updates without requiring the user to manually inspect everything.	Grounded in the AutoCheck/alert artifacts. Verification focuses on boundary cases (threshold equality, expiration date boundaries) and idempotency.
F-GL-AUTO	Auto-generated grocery list from AutoCheck Output	Keeps a grocery list updated based on priority triggers, reducing missed restocks and duplicate planning.	Consumes AutoCheck Output. Priority triggers (in order): (1) OUT_OF_STOCK, (2) EXPIRED, (3) LOW_STOCK, (4) FREQUENT & (LOW/OUT). Items already on the list are skipped. Actions are logged.
F-GL-USER	Grocery list review and editing flow	Users can review auto-added items, then remove/edit/add items manually; updated list is saved automatically.	Based on the grocery list user flow artifacts. Emphasizes user override and preventing clutter.
F-SPEND (optional scope if budgeting is in Milestone 1)	Spending capture and basic summaries	Allows household members to record spending and view simple period summaries (shared vs personal), supporting budget awareness.	Supports US-SB-* and US-RPT-* (basic reporting). Detailed finance flows can be deferred if needed.

NOTE	Feature F-GL-AUTO and F-GL-USER are based on the provided grocery list generator flow and user flow artifacts. Implementation details such as UI layout and backend

> scheduling are deferred to Section 2.3; here we capture only the user-visible capability and the trigger/priority semantics.

2.2.2. Domain Requirements

Requirements expressed in domain language (capabilities/outcomes), independent of UI/technology. Terminology follows the domain description: **Household**, **Household member**, **Location**, **Item type**, **Item instance**, and **Spending entry**.

2.2.2.1. Inventory integrity

- DR-1 (Barcode uniqueness): A single barcode shall map to exactly one **Item type**. The system shall not create a second item-type definition for an already-known barcode.
- DR-2 (Item instance validity): Every **Item instance** shall reference an existing **Item type** at the time it is recorded.
- DR-3 (Financial immutability): Removing an item from the tracked inventory (delete/archive/merge/split/adjust) shall not rewrite or delete associated **Spending entries**; financial history is immutable.

2.2.2.2. Shared-household rules

- DR-4 (Household administration): Each **Household** shall have exactly one household admin role at a time. Administrative responsibility shall be transferable before the current admin leaves.
- DR-4A (Invitation control): The system shall allow household participation to be extended through invitations governed by household permissions.
- DR-4B (Invitation lifecycle): An invitation shall have a defined lifecycle state such as active, accepted, declined, expired, or revoked.
- DR-4C (Revoked access): When a household member is removed from a household, their participation state shall change accordingly and access to that household's shared information shall no longer remain active.
- DR-5 (Membership independence): A system user may belong to multiple households, each with an independent membership role and membership state.
- DR-6 (Attribution retention): Spending entries shall remain attributed to the household member who made the purchase, even when inventory quantities are reorganized (merge/split) or when items are marked shared/personal.

2.2.2.3. Duplicate handling and intent preservation

- DR-7 (Short-window duplicates): If a duplicate item instance entry is detected within a short window (initially 30 seconds) with matching identity-relevant attributes, the tracked record shall aggregate quantity rather than create a separate entry (unless the user explicitly chooses to keep them separate).
- DR-8 (Ambiguity guard): If item identity is ambiguous (near-duplicate name/brand/variant without authoritative identifier), the system shall not auto-merge without explicit confirmation.
- DR-9 (Auditability of corrections): Inventory corrections (merge/split/adjust) shall be auditable

and reversible at the record level; the system shall preserve what changed, when, and why.

2.2.2.4. Reporting integrity (if reporting is in scope)

- DR-10 (Separation of concerns): Reports shall distinguish between inventory state (availability, locations, expiry) and spending history (amounts, dates, attribution); these shall not be conflated.
- DR-11 (Budget derivation): Budget/spending summaries shall be derived from spending entries within a defined budget period and shall not be altered by non-financial inventory corrections (e.g., discarding expired items).

2.2.2.5. Alerts and restock (Milestone 1 baseline)

- DR-12 (Threshold-driven low-stock): The system shall evaluate whether an item type is low-stock by comparing its current quantity to an item-specific threshold. When quantity falls at or below the threshold, the system shall flag the item type as low-stock and make this status visible to household members.

  Domain property behind this requirement: In the domain, low-stock is a judgment that an item is running low. This judgment may be qualitative or threshold-based. The requirement commits the system-to-be to threshold-based evaluation.

- DR-13 (Expiration status): The system shall derive the expiration status of an item instance when an expiration date is present. If the expiration date is missing or unreadable, the system shall treat expiration status as unknown and shall not generate false-expired conditions.
- DR-14 (Deduplicated alerting): The system shall deduplicate alerts and trigger them primarily on actionable state transitions, so that repeated checks without state change do not generate repeated alerts.

2.2.2.6. Duplicate prevention and data integrity

- DR-15 (Normalization before comparison): The system shall normalize **Item type** definitions before duplicate comparison and before insert, so that duplicate detection produces consistent and reproducible results.
- DR-16 (Spend attachment after duplicate resolution): The system shall finalize duplicate resolution for an item before creating any associated **Spending entry**, so that each spending entry is attached to exactly one resolved item type or household record without ambiguity.
- DR-17 (Single item type per barcode at persistence): At persistence, one barcode shall map to exactly one **Item type**; the system shall enforce this constraint before insert.

2.2.2.7. Rationale / trace (Milestone 1)

- DR-1..DR-3 support reliability of tracked inventory and financial records.
- DR-4..DR-6 support shared-household accountability and coordination.
- DR-7..DR-9 support data quality and conflict reduction during entry/corrections.
- DR-10..DR-11 support clear reporting and budget reasoning.

- DR-12..DR-14 support low-stock/expiry awareness with notification quality.
- DR-15..DR-17 support duplicate prevention and data integrity.

2.2.3. Interface Requirements

Interface requirements describe how phenomena and concepts from the domain are represented inside the system-to-be, how they are initialized, and how they are kept in sync when the real world changes.

These requirements were derived through elicitation techniques including semi-structured interviews with students and lightweight story mapping, which were used to identify important user interactions, edge cases, and system behaviors grounded in real household scenarios.

2.2.3.1. IR-01: Item Initialization

When a household member acquires a new item in the real world, they must have a way to create a corresponding representation inside the system.

The system shall provide a means for household members to add new items to their tracked inventory, including name, quantity/availability, category, location, ownership status (personal/shared), and optional expiration date.

Needs: N1, N2 | Goals: MG1, JG1, AG3 | Domain concepts: Item type, Item instance, Location

Questions this addresses: - How does an item first appear in the system? - What information is needed to represent what was acquired? - How does the system learn an item exists?

2.2.3.2. IR-02: Item Update on Consumption

When a household member consumes part of an item in the real world, the tracked representation must be updated to reflect new availability.

The system shall provide a means for household members to record consumption by decreasing recorded quantity/availability or marking an item as finished/out-of-stock.

Needs: N1, N2 | Goals: MG1, MG3, JG1 | Domain concepts: Item instance, Quantity, Consumption

Questions this addresses: - How does the system learn an item was partially/fully consumed? - How is tracked availability kept aligned with physical reality? - What happens when an item is finished?

2.2.3.3. IR-03: Ownership Specification

In shared households, members need to know whether an item is personal or shared.

The system shall provide a means to mark an item as personal or shared, and if personal, associate it with a household member.

Needs: N4 | Goals: SG3, JG3 | Domain concepts: Household member, Item instance, Membership

Questions this addresses: - How does the system distinguish personal vs shared items? - Who can

see/act on which items? - How are conflicts over personal items reduced?

2.2.3.4. IR-04: Low-Stock Threshold Definition

Households use different “low” definitions depending on the item. This requirement addresses the interface aspect: providing a way to enter the threshold when creating an item, and a way to change the threshold when editing the item. The storage and evaluation of thresholds is addressed by DR-12.

The system shall provide a means to set low-stock thresholds for individual items. This includes: a threshold field on the item creation screen, a threshold field on the item edit screen, and optionally, a quick-edit affordance when an item is flagged as low-stock.

Needs: N2 | Goals: MG3, JG1, AG1 | Domain concepts: Low-stock threshold, Item type

Questions this addresses: - How does the system know what “low” means for different items? - Can members customize thresholds to match their routines? - What happens when no threshold is provided?

2.2.3.5. IR-05: Expiration Date Entry

Some items have expiration dates that must be captured to support expiration awareness.

The system shall provide a means to enter and edit expiration dates for items that expire; items without expiration dates must be supported.

Needs: N2 | Goals: MG2, JG2 | Domain concepts: Item instance, Expiration date

Questions this addresses: - How does the system learn when an item expires? - Is expiration required or optional? - How are items without expiration handled (no false confidence)?

2.2.3.6. IR-06: Purchase Logging

When a household member spends money on household items, that transaction must be representable in the system.

The system shall provide a means to log purchases, including items bought (or categories), total cost, date of purchase, and whether the purchase was personal or shared.

Needs: N5 | Goals: SG1, AG2 | Domain concepts: Spending entry, Spending category

Questions this addresses: - How does spending information enter the system? - How can purchases be linked to items or categories? - How is shared vs personal spending distinguished?

2.2.3.7. IR-07: Cost Splitting

For shared purchases, costs may need to be divided among household members.

The system shall provide a means to split shared purchase costs among selected household members and track who owes what.

Needs: N5 | Goals: SG2, AG2 | Domain concepts: Spending entry, Household member, Membership

Questions this addresses: - How are shared costs divided? - How does the system know who participates in a split? - How can disputes be reduced via clarity and traceability?

2.2.3.8. IR-08: Dashboard Metric Overview

Household members need a high-level overview without searching through details.

The system shall provide a dashboard view summarizing key metrics such as total items, low-stock items, expiring/expired items, and current-period spending.

Needs: N1, N3 | Goals: MG1, AG1, AG3 | Domain concepts: Inventory, Budget status, Low-stock threshold, Expiration date

Questions this addresses: - How does a member quickly assess what needs attention? - Which metrics matter most at a glance? - How are different tracked phenomena summarized consistently?

2.2.3.9. Appendix A: Edge-Case Interface Requirements

The following edge-case interface requirements are recognized but deferred for detailed specification in later milestones. They are captured in this appendix, separate from core IRs, to avoid losing scope awareness and to support future traceability.

IR-EC-A: Near-Duplicate Variant Disambiguation

When a user adds an item whose name matches an existing item type but whose size, variant, or brand differs, the system must present the conflicting attribute highlighted and require the user to choose: "This is a different variant" or "Use existing item type." Same-name/different-size entries shall be treated as separate item type definitions by default and must not be auto-merged.

IR-EC-B: Stock Entry Separation by Expiration and Location

When a user attempts to merge two item instances for the same item type that have different expiration dates or different storage locations, the system must warn that merging will lose expiration/location tracking. The default action must be "Keep separate"; merging must require explicit confirmation.

IR-EC-C: Import Idempotency and Duplicate Spend Prevention

When a user uploads a CSV import file that was previously imported, the system must default to "Review before apply" and present a summary of potential duplicate transactions. Per-row, if a transaction fingerprint matches a previously imported record, the system must skip spend event creation by default and require an explicit "Import anyway" override.

IR-EC-D: Inventory Correction Without Budget Impact

When a user deletes or merges an inventory entry that has associated spend events, the system must present two independent controls: "Remove from inventory" and "Also mark purchase as mistake — exclude from budget". Inventory correction must never delete spend events automatically. Both actions must be logged separately in the audit trail.

IR-EC-E: Refund and Return Flow

When a user returns a purchased item, the system must provide a refund flow that: decrements inventory quantity, creates a negative spend entry linked to the original where possible, and requires amount refunded, date, and an optional link to the original receipt or spend event.

IR-EC-F: Waste and Shrinkage Adjustment

When a user removes expired, lost, or consumed items from inventory outside of a normal purchase/return flow, the system must: allow inventory adjustment without altering any spend event, require an adjustment reason, and offer an optional "Add to waste report" checkbox. Waste metrics must be reported separately from spending.

IR-EC-G: Shared Household Budget Attribution Prompt

When a user adds an item in a shared inventory, the system must prompt "Who bought this?" with the current user as default. If the user selects another household member, the system must require confirmation. Cross-user inventory merges are permitted for quantity counts, but each spend event must retain its original buyer attribution.

IR-EC-H: Undo for Auto-Merge Actions

Every auto-merge of inventory quantities must be reversible within 10 seconds via an undo control. Undo must revert quantity changes and record the undo in the audit log.

IR-EC-I: Audit Trail for Edge-Case Resolutions

Every edge-case resolution that modifies inventory state must generate a log entry containing at minimum: timestamp, user, household, source, action type, item type reference, item instance reference, spending entry reference, reason, and a reference to the previous state. This log must be accessible to the household admin.

2.2.4. Machine Requirements

Machine requirements define measurable quality properties for Milestone 2 implementation. Since the prototype is under development, these requirements establish targets that will be verified upon completion. Where specific thresholds are not yet known, the requirement defines what will be measured and how success will be evaluated.

Each requirement includes traceability to relevant Domain Requirements (DR) where applicable.

2.2.4.1. MR-01: Availability

The system shall remain operational during supervised usability testing. No unplanned downtime requiring restart shall occur during testing sessions.

Trace : DR-12 (threshold-driven low-stock), DR-13 (expiration status)

Verification : Session logs; test coordinator observation.

2.2.4.2. MR-02: Response Time

The system shall respond quickly enough that household members can complete inventory tasks without waiting. During usability walkthroughs: - No participant shall abandon a task due to perceived slowness - All participants shall complete lookup tasks while simulating shopping (standing, holding phone) - Item creation shall be completable in under 1 minute (consistent with US-INV-01)

Response time measurements will be collected during walkthroughs to establish baseline for future milestones.

Trace : DR-7 (short-window duplicates), DR-12 (threshold-driven low-stock)

Verification : Usability walkthrough observations; timing recorded for each task.

2.2.4.3. MR-03: Offline Operation

The mobile client shall support viewing inventory while offline. Adding items and updating quantities while offline is deferred to post-Milestone 2.

Trace : DR-14 (deduplicated alerting)

Verification : Integration tests with connectivity simulation.

2.2.4.4. MR-04: Synchronization

When multiple household members use the same inventory, changes shall be propagated to all members in real time via Supabase Realtime. Changes shall appear without requiring manual refresh.

Trace : DR-4 (household administration), DR-5 (membership independence)

Verification : Automated tests verifying real-time delivery; manual tests with 2-3 concurrent users confirming changes appear instantly.

2.2.4.5. MR-05: Security and Privacy

Only household members shall have access to their respective household inventories. A user belonging to multiple households shall have access only to the household for which they are a member. Access revocation for a removed member shall take effect upon their next login attempt for that specific household.

Trace : DR-4 (household administration), DR-5 (membership independence), DR-6 (attribution retention)

Verification : Access control tests with test accounts spanning multiple households.

2.2.4.6. MR-06: Auditability

Quantity changes shall be logged with timestamp, user ID, item, and previous value.

Trace : DR-8 (ambiguity guard), DR-9 (auditability of corrections)

Verification : Schema validation; manual log inspection.

2.2.4.7. MR-07: Notification Quality

Repeated automated checks without state changes shall not generate duplicate alerts during testing.

Trace : DR-14 (deduplicated alerting)

Verification : Integration tests with repeated triggers.

2.2.4.8. MR-08: Barcode Scanning and Fallback

The mobile client shall use the device camera to scan barcodes for item identification. When a barcode is scanned, the system shall attempt to match it to an existing item type. If no match is found, the system shall allow manual entry without blocking the workflow. If the barcode is unreadable or scanning is unavailable, manual entry shall remain the primary fallback.

NOTE	Barcode uniqueness is a domain requirement, not a machine requirement. This MR addresses the interface/device aspect: using the camera to capture barcodes and falling back gracefully when scanning fails.

Trace : DR-1, DR-13

Verification : Manual tests with valid barcode, invalid barcode, and scanning unavailable scenarios.

2.2.4.9. Measurement Plan for Milestone 2 Completion

The following measurements will be collected during Milestone 2 validation activities to establish baseline for future milestones:

What to Measure	How	When
Response times	Record during usability walkthroughs	Post-prototype
Sync behavior	Verify real-time propagation via Supabase	During multi-user testing

What to Measure	How	When
Error occurrences	Log all exceptions during testing	Throughout testing
User perception	Participant feedback on speed/friction	During walkthroughs
Barcode scanning success rate	Track match vs manual entry ratio	During walkthroughs

These measurements will inform quantitative machine requirements for subsequent milestones.

2.2.4.10. Out of Scope for Milestone 2

Quantitative targets cannot be established without a prototype. These will be defined in Milestone 3 based on measurements collected during Milestone 2 validation.

2.2.5. Shared-Household Invitations and Access Changes

In shared households, invitations are the mechanism by which participation is extended, updated, or removed. This section defines the requirement-level behavior needed to support household access changes without introducing implementation-specific onboarding details.

A household invitation is a temporary request that allows a user to join a household context. Each invitation identifies a sender, a recipient, a target household, a status, and an expiration time.

2.2.5.1. Invitation permissions

Invitation permissions determine which household members may invite others into a household. By default, invitation authority belongs to the household admin role, though later requirements may allow additional role-based permissions.

2.2.5.2. Invitation lifecycle

An invitation may move through the following states:

- **Sent** — the invitation has been created and is awaiting a response.
- **Accepted** — the recipient joins the household and the invitation becomes inactive.
- **Declined** — the recipient rejects the invitation and the invitation becomes inactive.
- **Expired** — the invitation is no longer valid because its time window has passed.
- **Revoked** — the sender or household admin invalidates the invitation before acceptance.

These states matter because they determine whether household participation becomes active and whether access to shared information should be granted or removed.

2.2.5.3. Core rules

- Invitations to the same recipient for the same household should not remain duplicated in an active state.
- Expired or revoked invitations cannot be reused.
- Accepting an invitation grants household access according to the assigned role.
- Declining or revoking an invitation does not grant access.
- When a household member is removed, their access to shared household information must no longer remain active.

2.2.5.4. Shared access behavior

Once access is granted, the shared household inventory becomes visible according to the member's assigned role. Depending on the role definition in later requirements, a member may have view-only access, update privileges, or administrative responsibility.

Household administration may also include viewing current members, resending invitations, changing roles where supported, and revoking access when participation changes.

2.3. Implemented End-to-End Features

This section documents the minimal viable set of high-value features that have been fully implemented in the current release. For each feature, the end-to-end user flow is mapped from initial input through processing to final output. This enables reviewers to verify functionality and understand the core system behavior without needing to compile or run the application.

Status Legend: Completed | In Progress | Planned

2.3.1. Feature 1: Core Inventory Management (F-INV)

User Value: Eliminates tedious manual data entry by enabling quick item capture via barcode scanning with fallback manual entry. Reduces onboarding friction for household managers like Andrea.

Primary Users: Household managers, kitchen coordinators

End-to-End User Flow:

Table 1. Input / Process / Output Flow

Input	Processing	Output
User taps "Add Item" floating action button	System opens camera intent and initializes barcode scanner UI	Camera preview displayed with scanning overlay

Input	Processing	Output
User scans product barcode OR manually enters product name	- Barcode detected: query product database (local cache + remote API) - Product found: auto-populate (name, brand, image, default quantity) - Product not found: show manual entry form fallback	Item details form pre-filled or blank for manual entry
User sets expiration date, quantity, location, and taps "Save"	- Data validation (expiration date ≥ today, quantity > 0) - Insert into Supabase `products` table - Trigger Bloc/Cubit state update - Stream listener broadcasts real-time update to inventory list	Item appears instantly in Core Inventory list; list view refreshes via Flutter Bloc state management
User views inventory list (sorted by category/status)	Supabase real-time stream continuously syncs inventory state across all connected listeners	Live inventory display with current quantities, expiration status, and item metadata

Detailed User Scenario (Andrea's Onboarding): 1. Andrea opens the app and navigates to the "Inventory" tab 2. She taps the floating action button (FAB) to add her first item 3. Camera opens; she scans a pasta box 4. System recognizes it as "Barilla Penne Rigate" with product image 5. Auto-populated form shows: name, brand, category, image 6. She adjusts quantity to 2 boxes, sets expiration date to 2026-12-31 7. She taps "Save" and the item appears in the inventory list

Components Involved: - **Presentation:** `AddItemPage`, `InventoryPage`, `ItemForm` widgets - **State Management:** `InventoryCubit`, `InventoryState` - **Data Layer:** `ProductSupabaseDataSource`, `InventorySupabaseDataSource`, `ProductRepositoryImpl` - **Domain:** `AddInventoryItem`, `GetInventoryItems` use cases - **External:** Barcode scanner plugin, Supabase Realtime streams

Visual Evidence:

[Barcode Scanner UI] | *Barcode Scanning and Manual Entry/scanner.png* *Figure 1. Live Barcode Scanning*	[Core Inventory List] | *screen-designs/Core-Inventory-Designs/core_inventory_mockups.png* *Figure 2. Inventory List View*

Key Implementation Details: - **Real-Time Sync:** Uses Supabase Postgres Realtime subscriptions to push updates to all clients instantly - **Validation:** Client-side validation ensures data integrity before submission; server-side triggers enforce constraints - **Barcode Fallback:** If barcode scanning fails or is unavailable, users can manually search and select from a product catalog - **Category Management:** Items are automatically categorized based on product metadata; users can override if needed

Completion Status: Completed — Core add/view/edit operations fully functional. Barcode scanning integrated with fallback manual entry.

2.3.2. Feature 2: Automated Expiration Alerts & Notifications (F-ALERT + F-AUTOCHECK)

User Value: Prevents food waste by proactively alerting users before items spoil. Removes the mental burden of manual fridge checks, especially critical for busy users like Mateo.

Primary Users: All household members (push notifications), household admin (alert configuration)

End-to-End User Flow:

Table 2. Alert Generation & Delivery Pipeline

Input	Processing	Output
Supabase Cron job triggers daily at 08:00 UTC	1. Query all non-archived items in `products` table 2. Calculate days until expiration for each item 3. Apply alert thresholds (e.g., notify if expires in ≤ 3 days) 4. Deduplicate: skip items already notified in last 24h (check `alert_log`) 5. Build payload with expiring items	List of items meeting alert criteria, ready for notification
Alert payload generated (item name, expiration date, action URL)	1. Validate user FCM/APNs token availability 2. Format payload with deep link to Alerts Center 3. Send via Firebase Cloud Messaging (FCM) or Apple Push Notification service (APNs)	Push notification delivered to user's device
User taps push notification on mobile device	App route handles deep link and navigates to the "Alerts Center" tab to highlight expiring items	Alerts Center screen displays: - List of expiring items sorted by urgency - Recommended actions (consume, donate, discard, extend date) - Quick action buttons
User taps "Mark as Consumed" or "Extend Expiration" for an item	1. If Mark Consumed: delete or reduce quantity to 0 2. If Extend: show date picker, update expiration date 3. Trigger state update via Bloc 4. Log action in `alert_log` to prevent duplicate notifications	Item removed from or updated in inventory; alert cleared; confirmation shown

Detailed User Scenario (Mateo's Morning Alert): 1. Mateo receives a push notification: "2 items expiring today or tomorrow" 2. Notification shows: "Milk (expires today)", "Yogurt (expires tomorrow)" 3. He taps the notification and the app opens and navigates to Alerts Center 4. He sees: - **Milk** | Expires: Today | Action: Consume Now / Discard - **Yogurt** | Expires: Tomorrow | Action: Consume Today / Extend to [date] 5. He taps "Consume Now" next to Milk and the item is marked consumed with quantity set to 0 6. For Yogurt, he taps "Extend" and the date picker appears; he extends it to 2 days out 7. Alerts list auto-refreshes; both items are now off the alert list

Components Involved: - **Backend:** Supabase Cron Edge Function (TypeScript), Postgres triggers -

Frontend Presentation: Alerts Center screen, push notification handler - **State Management:** Alert Cubit (if implemented) or real-time stream listener - **Data Layer:** Alert repository, alert log data source - **Domain:** `AutomatedCheckExpiration`, `AutomatedCheckStock` use cases - **External:** Firebase Cloud Messaging (FCM), Apple Push Notification (APNs)

Visual Evidence:

[Push Notification Flow] | screen-designs/Alerts-&-Notifications-Screen/push-notifications-flow-ui.png *Figure 3. Push Notification & Deep Link*	[Alerts Center UI] | screen-designs/Alerts-&-Notifications-Screen/alerts-center-mockup.png *Figure 4. Alerts Center Screen*

Key Implementation Details: - **Deduplication Logic:** Alert log stored in Supabase prevents notification spam; same item is not re-notified within 24 hours unless status changes - **Threshold Flexibility:** Thresholds are configurable per household (e.g., some families want 7-day warnings, others 1-day) - **Timezone Handling:** Cron job runs in UTC; expiration date comparison uses user's local timezone - **Offline Handling:** If user is offline, Android/iOS queues notification locally; it displays when device reconnects

Completion Status: Completed — Alert generation logic, deduplication, and notification dispatch fully implemented. Alerts screen integrated with deep-link handling.

2.3.3. Feature 3: Smart Grocery List Management (F-GL-AUTO + F-GL-USER)

User Value: Closes the inventory lifecycle loop. Auto-generates shopping lists based on inventory state, then enables quick shopping trip management (check off items, restock on return).

Primary Users: Shoppers, household managers

End-to-End User Flow:

Table 3. Grocery List Generation & Trip Management

Input	Processing	Output
Automated daily check (from Feature 2) runs and identifies inventory gaps Example: Milk quantity = 0, Bread expired 2 days ago, Pasta stock below threshold	1. Process AutoCheck output (items with status: OUT_OF_STOCK, EXPIRED, LOW_STOCK) 2. Apply priority order: - Priority 1: OUT_OF_STOCK items - Priority 2: EXPIRED items - Priority 3: LOW_STOCK items - Priority 4: Frequently used + low/out 3. Skip items already on active grocery list 4. Log auto-additions to `grocery_list_log`	Grocery list auto-updated with prioritized items (e.g., Milk [OUT_OF_STOCK], Bread [EXPIRED])

Input	Processing	Output
User navigates to "My Grocery List" screen	App fetches active grocery list from Supabase; displays items grouped by category Options shown: - Checkbox to mark item as "in cart" - Edit quantity / notes - Remove item - Add custom item (manual entry)	Grocery list displayed with auto-generated items + any user-added custom items
User is at supermarket; checks off items as they enter cart Then, user completes shopping trip and taps "Finalize Grocery Run"	1. For each checked item: - Prompt for quantity bought (default = inventory gap amount) - Prompt for expiration date (if perishable) 2. Collect receipt info (optional: total spent, payment method) 3. Validate entries	Dialog/form shown for final confirmation + quantity/expiration entry
User confirms and taps "Save & Restock"	1. For each item, update/insert into Core Inventory with new quantities and expiration dates 2. Clear checked items from grocery list (or archive) 3. Log restocking action in `grocery_list_log` 4. Trigger inventory state update (Bloc)	Updated inventory displayed in Core Inventory screen; grocery list cleared of checked items; confirmation message shown

Detailed User Scenario (Andrea's Weekly Shopping Trip): 1. Andrea opens the app on Sunday; she sees the Grocery List tab shows 6 auto-generated items: - Milk (OUT_OF_STOCK) — Qty: 1 - Bread (EXPIRED) — Qty: 1 - Pasta (LOW_STOCK) — Qty: 2 - Eggs (LOW_STOCK) — Qty: 3 - Cheese (added manually by Andrea) - Butter (added manually by Andrea) 2. Andrea heads to the supermarket 3. She checks off items as she places them in her cart: - Milk (1 liter) - Bread (2 loaves) - Pasta (2 boxes) - Eggs (1 dozen) - Cheese (500g) - Butter (500g) 4. At checkout, she taps "Finalize Grocery Run" 5. System shows a summary dialog: `` Review your purchases: - Milk [1 box] | Expires: 2026-04-20 - Bread [2 loaves] | Expires: 2026-04-08 - Pasta [2 boxes] | Expires: 2027-04-07 - Eggs [1 dozen] | Expires: 2026-05-01 - Cheese [500g] | Expires: 2026-06-15 - Butter [500g] | Expires: 2026-05-01 [Spent: $--] [Save & Restock] `` 6. She taps "Save & Restock" 7. App updates Core Inventory with new items and quantities 8. Grocery list is cleared (checked items archived) 9. Confirmation: "6 items restocked! Your inventory is up to date."

Components Involved: - **Presentation:** `MyGroceryListPage`, `GroceryListTile`, `AddCustomItemPage` widgets - **State Management:** `GroceryListCubit` (if implemented) or stream-based updates - **Data Layer:** Grocery list repositories and data sources, expense tracking - **Domain:** Grocery list use cases for generation and update - **Auto-Generation:** Scheduled function that runs daily AutoCheck output processing

Visual Evidence:

[My Grocery List] | *../requirements/features/grocery-list/design/images/My Grocery list UI.png*	[Add Custom Item] | *../requirements/features/grocery-list/design/images/Add Costume item UI.png*	[Shopping Checkout] | *../requirements/features/grocery-list/design/images/grocery-list-during-shopping-trip.png*
Figure 5. Grocery List View	*Figure 6. Add Custom Item*	*Figure 7. Finalize & Restock*

Key Implementation Details: - **Priority Queue:** Auto-generation respects priority order to prevent overwhelming users with too many items - **Prevent Duplicates:** Items already on the active list are not re-added by AutoCheck - **Custom Items:** Users can manually add items (e.g., "Olive Oil" if not in product database) - **Quantity Flexibility:** When restocking, users can adjust quantities based on actual needs/availability - **Expense Integration:** Optional capture of spending per trip (supports budgeting feature)

Completion Status: Completed — Auto-generation via priority queue, user review/edit flow, and restocking (update to core inventory) all implemented.

2.3.4. Feature 4: Shared Inventory & Household Management (F-SHARING)

User Value: Enables multiple household members to collaborate on inventory and see shared items, roles, and responsibilities.

Primary Users: All household members

Current Status: - Household creation & member invitation: Implemented - Role-based item visibility: In Progress - Shared vs. personal items: In Progress

End-to-End User Flow (As Currently Implemented):

Table 4. Household Setup & Member Invitation

Input	Processing	Output
User creates a new household during onboarding User enters: Household name, description	1. Validate input (non-empty name, max 100 chars) 2. Insert into `households` table with user as owner 3. Create initial role assignment (user = OWNER) 4. Initialize household settings (notification preferences, etc.)	Household created; user becomes owner; household ID returned
Owner navigates to Settings and selects Invite Members Owner enters: Roommate email, role (MEMBER, ADMIN, VIEWER)	1. Generate unique invite token 2. Send email with invite link + token 3. Store invite record in `invitations` table 4. Log invitation action	Invite email sent to roommate with deep link to accept/decline
Roommate receives email and taps invite link App opens to invitation screen showing: Household name, owner, role, permissions summary Roommate taps "Accept"	1. Validate invite token (not expired, not already used) 2. Create user in `household_members` table with provided role 3. Mark invitation as accepted 4. Trigger Bloc update for household members list	Roommate added to household; can now see shared items; confirmation shown

Components Involved: - **Presentation:** Household settings, invite screen, member list view - **State Management:** Household Cubit (if implemented) - **Data Layer:** Household repositories, invitation

data source - **Domain:** Household management use cases - **External:** Email service for invitations

Visual Evidence:

image::Invite-Roomate-Screen-UI/invite_roommates_mockup.png[Invite Roommates, width=100%, title="Send Invite UI"]	image::View-roommate-invitation-screen/invitation_acceptance_mockup.png[Accept Invite, width=100%, title="Accept Invite UI"]

Completion Status: In Progress — Household creation and member invitation fully implemented. Role-based item filtering and shared vs. personal item logic progressing.

2.3.5. Feature 5: Reporting & Insights (F-SPEND / Reports)

User Value: Provides visibility into spending patterns, inventory trends, and usage analytics to support budget awareness and meal planning.

Primary Users: Household managers, budget-conscious users

Current Implemented Reports: 1. **Inventory Stock Report** — Items by category, stock levels, expiration status 2. **Expenditure Report** — Spending summary by time period, household member

End-to-End User Flow (Stock Report):

Table 5. Inventory Stock Report Generation

Input	Processing	Output
User navigates to Reports and selects Inventory Stock Report User selects date range (optional)	1. Query `products` table for all items in household 2. Calculate stock levels (full, halfway, low, empty) 3. Identify expiring items (next 7 days) 4. Group by category 5. Generate summary stats (total items, critical items, etc.)	Report view populated with: - Summary cards (total items, by stock level, expiring count) - Grouped table (category, items, stock status)
User taps category to expand	Filter/expand logic in UI	Items in category displayed with details (quantity, threshold, expiration)
User taps "Export as PDF"	1. Fetch data 2. Render PDF using pdf library with charts/tables 3. Save to device storage	PDF file generated and ready to share/print

Detailed User Scenario (Manager's Weekly Review): 1. Andrea opens Reports and selects Inventory Stock Report 2. She sees summary: "28 items total | 18 adequate | 4 low stock | 2 expiring this week" 3. She expands categories: - **Dairy:** Milk (LOW), Yogurt (ADEQUATE), Cheese (ADEQUATE) - **Grains:** Bread (ADEQUATE), Pasta (ADEQUATE), Rice (EMPTY) - **Proteins:** Eggs (ADEQUATE) 4. She taps "Export as PDF" to share with household and groceries list 5. PDF downloaded and ready to view/print

Components Involved: - **Presentation:** Report screens, charts/tables, PDF preview - **State**

Management: Report Cubit - **Data Layer:** Report repositories - **Domain:** Report generation use cases - **External:** PDF library, file sharing APIs

Completion Status: In Progress — Basic stock reports implemented; expenditure reports and analytics features progressing.

2.3.6. Summary Table: MVP Feature Completion

Feature ID	Feature Name	Status	Key Components
F-INV	Inventory Management (Add/View/Edit items)	Completed	AddItemPage, InventoryCubit, ProductDataSource, Barcode Scanner
F-ALERT	Alert Generation & Notifications	Completed	AlertsCenterScreen, FirebaseMessaging, Supabase Cron, DeepLink Handler
F-AUTOCHECK	Automated Daily Inventory Check	Completed	AutoCheckFunction (Edge), Threshold Logic, Notification Dispatch
F-GL-AUTO	Auto-Generated Grocery List	Completed	GroceryListGeneration (Scheduled), Priority Queue, Deduplication
F-GL-USER	Grocery List Review & Restocking	Completed	MyGroceryListPage, RestockingFlow, Quantity/Expiration Input
F-SHARING	Shared Inventory & Household Management	In Progress	HouseholdSettings, InvitationFlow, RoleManagement
F-SPEND	Spending & Budget Reporting	In Progress	ReportScreens, Charts, PDF Export, ExpenseTracking

---

2.3.7. Architecture & Technical Stack

Frontend Stack: - **Framework:** Flutter 3.29.0 with Dart 3.7.0 - **State Management:** Flutter Bloc (Cubit for simple states) - **Navigation:** GoRouter with deep-link support - **UI Components:** Material 3 design, custom Material/Cupertino widgets - **Local Storage:** Shared Preferences (settings), cached product data - **Notifications:** Firebase Cloud Messaging (FCM) + platform-specific handling

Backend Stack: - **Database:** Supabase PostgreSQL with Realtime subscriptions - **Authentication:** Supabase GoTrue (email/password, social) - **Backend Functions:** Supabase Edge Functions (TypeScript) for cron jobs - **File Storage:** Supabase Storage (product images, receipts) - **APIs:**

Barcode lookup service (for product data enrichment)

CI/CD & Deployment: - Version control: Git + GitHub - Testing: Dart test framework, mocktail for mocking - Linting: flutter_lints, Dart analyzer

---

2.3.8. Testing & Verification Strategy

Each completed feature has been verified through:

1. **Unit Tests:** Business logic (use cases, repositories) with mock data sources
2. **Widget Tests:** UI components in isolation (forms, lists, dialogs)
3. **Integration Tests:** End-to-end flows on emulator/physical device
4. **Manual QA:** Cross-device testing (Android, iOS), network conditions (offline, slow)

Example verification for F-INV (Inventory Management): - Add item via barcode scanning - Add item via manual entry fallback - Edit existing item (quantity, expiration, category) - Delete item - Real-time list refresh on other device when item added - Validation (no negative quantities, future expiration dates)

2.3.9. Known Limitations & Future Work

Current Limitations: 1. Role-based item visibility not yet enforced (all shared items visible to all members) 2. Budget alerts not yet implemented (spending-based notifications) 3. Complex searches (filter by location, owner, expiration range) deferred to post-MVP 4. Barcode scanning limited to common products; many items require manual entry

Future Features (Post-MVP): 1. **AI-Powered Meal Planning:** Suggest recipes based on current inventory 2. **Smart Notifications:** ML-based expiration risk scoring (some items spoil faster than labels suggest) 3. **Inventory Forecasting:** Predict when items will run out based on consumption patterns 4. **Family Analytics:** Who consumes what, eating pattern insights 5. **Integration with grocery delivery services:** Direct ordering from grocery list

---

3. Analytic Part

3.1. Concept Analysis

3.1.1. Domain Concept Analysis

This section derives domain concepts from the domain rough sketch (interviews, observations, edge situations). It documents how specific domain statements lead to abstractions (concepts), identifies potential term clashes, and defines relationships and invariants that hold in the domain independent of any system-to-be.

3.1.1.1. Starting point: raw statements held side-by-side

The following statements (from the rough sketch) are used as anchors for concept formation:

- Shared apartment: "If it is not on our fridge note, we assume we are out."
- Shared apartment: "Sometimes I check the pantry before leaving, but by the time I get to Walmart I forgot what we needed."
- Shared apartment: "We have rice in two places. One bag in the kitchen cabinet and another in a plastic bin in the hallway."
- Shared apartment: "Milk is usually bought twice in the same week at least once per month."
- Shared apartment: When asked how they know they are 'low': "when it feels light" or "when we open the backup." (and the backup opening is not recorded)
- Solo student: "If it takes more than a few seconds to log, I won't do it."
- Solo student: buys in bulk; yogurts sometimes expire; estimated 3–5 thrown away last semester.
- Household owner: child sometimes uses last item without saying; toilet paper runs out unexpectedly.
- Household owner: medicines kept separately; expiration dates important; thrown away expired medicine without realizing there was a second unopened box.
- Household owner: wants to remove access when someone moves out.
- Student athlete: consumption changes by context ("exam weeks"); buys based on memory of how fast it ran out last time.
- Brainstorming: people describe items as "full", "almost empty", "backup" (rarely with numbers).
- Edge situations: unreadable/missing expiration date; multiple expiration dates within a bulk package; backup stored in car trunk and forgotten.

These statements motivate the concepts below.

3.1.1.2. Concept formation: from statements to abstractions

We derive concepts by asking: (1) what phenomena exist in the domain, (2) what information households use to act, and (3) what changes over time.

Examples of abstraction:

- "If it is not on our fridge note, we assume we are out." → **Tracked representation** (note/list) vs **reality** (actual item availability), and **assumption under uncertainty**.
- "Rice in two places" → item storage **location**, and the possibility of multiple locations for the same item.
- "when it feels light" / "open the backup" → **low-stock judgment** that is qualitative and strategy-based, and **backup** as a household practice.
- "Milk bought twice in the same week" → **duplicate purchase** as a coordination failure, often arising from missing shared understanding.
- "3–5 yogurts thrown away last semester" / medicines expired with unopened box → **expiration** and **batches** (same item type can exist in multiple packages with different expiration dates).
- "remove access when someone moves out" → **membership** and **access to shared household information** (domain concept: who belongs to the household at a given time).

3.1.1.3. Core domain concepts (derived)

Concept	Definition (domain-level)
Household	A living unit (dorm, apartment, shared house) in which one or more people manage household essentials.
Household member	A person who currently belongs to a household and participates in consuming, replenishing, or coordinating around items.
Household item	A physical good kept for everyday use (e.g., milk, rice, toilet paper, soap, medicine).
Item type	The kind of item (e.g., "milk") independent of any specific package.
Item instance / package	A concrete package/unit of an item type (e.g., a specific milk carton, a specific box of medicine).
Inventory	The collection of household items that exist within a household at a given time.
Tracked inventory record	A representation of item availability used by household members (e.g., fridge note, mental model, list). This may diverge from the real inventory.
Location (storage place)	A physical place where items are stored (e.g., kitchen cabinet, hallway bin, bathroom closet, car trunk).
Availability	A household's current "how much is left" understanding, often qualitative ("full", "almost empty", "backup").
Quantity (optional formalization)	A more precise measure of availability (counts/units). The domain can operate without exact numbers.
Low-stock judgment	A decision that an item is running low, derived from strategies like "feels light", "opened backup", or a numeric threshold.

Concept	Definition (domain-level)
Backup supply	A household practice of keeping a spare instance/package to reduce the risk of running out.
Consumption	Use of items over time (may vary by context such as exams/guests).
Replenishment (restock)	Acquiring more of an item after consumption reduces availability.
Expiration date	A date after which an item instance/package is considered expired or undesirable.
Item batch (optional)	A set of item instances with the same expiration date (useful when bulk packages contain multiple expirations).
Coordination failure	A situation where household members act on inconsistent information (e.g., duplicate purchase, unexpected shortage).

3.1.1.4. Term clashes and clarifications

To prevent inconsistent usage across documentation, we clarify common ambiguous terms:

- **Inventory** (real-world items in the household) vs **Tracked inventory record** (note/list/memory). The rough sketch shows these diverge (e.g., “not on fridge note → assume out”).
- **Item type** (“milk”) vs **Item instance/package** (a specific carton). This matters for expiration and “second unopened box”.
- **Low-stock** is not inherently numeric; in the domain it is often qualitative (“feels light”, “open backup”).
- **Backup** is a domain concept (household practice), not an application feature.
- **Household membership** changes over time (“remove access when someone moves out”): membership is a domain phenomenon; access control is a system concern derived from it.

3.1.1.5. Relationships (domain model)

Concept A	Concept B	Relationship
Household	Household member	A household has one or more members; membership may change over time.
Household	Inventory	A household has an inventory (items that exist in the household).
Household item	Item type	Each household item corresponds to an item type.
Item type	Item instance/package	An item type may have multiple item instances/packages in the household at once (bulk buying, backup supply).

Concept A	Concept B	Relationship
Item instance/package	Location	Each item instance is stored in one location at a time; the same item type may be split across locations.
Item instance/package	Expiration date	Some item instances have an expiration date; others do not.
Household member	Consumption / replenishment	Members consume and replenish items; consumption rates vary by context (exam weeks, guests).
Tracked inventory record	Inventory (reality)	Tracked records approximate reality and may be incomplete, outdated, or wrong.
Household	Coordination failure	Coordination failures arise when members act on inconsistent or outdated tracked records.

3.1.1.6. Events, actions, and behaviors (domain-level)

- Events (instantaneous state changes):
  - An item becomes **out** (unexpected shortage).
  - An item becomes **low** (as judged by a strategy such as “backup opened”).
  - An item expires (or is discovered expired).
- Actions (intentional acts):
  - Check availability (pantry check, fridge note check).
  - Consume an item (use the last roll, eat snacks faster during exams).
  - Replenish an item (buy milk, buy bulk yogurt).
- Behaviors (multi-step ongoing processes):
  - Household maintenance cycle: consume → notice low/out → decide purchase → replenish → store → repeat.
  - Shared-household coordination: negotiate who buys what, or fail and duplicate-buy.

3.1.1.7. Invariants (domain rules)

These invariants describe stable properties of the domain:

- I1: A household has at least one member at any time it is considered active.
- I2: An item instance/package, if it exists, is stored in some location (even if the location is forgotten, e.g., car trunk).
- I3: Not all household judgments require numeric quantities; qualitative availability (“full”, “almost empty”, “backup”) is valid domain information.
- I4: If an item instance has an expiration date, it can become expired without being consumed, creating waste.
- I5: A household may possess multiple instances of the same item type simultaneously (bulk

buying, backups).

- I6: Tracked records can diverge from reality; decisions may be made based on assumptions under uncertainty ("not on note → assume out").

3.1.1.8. Edge cases and risks (from rough sketch)

- E1: Same item type stored in multiple locations (rice in cabinet + hallway bin) → risk of undercounting or duplicate purchase.
- E2: Backup supply opened but not recorded → low-stock event is not propagated to shared understanding.
- E3: Consumption rate varies by context (exam weeks, guests) → replenishment based on "last time" memory becomes unreliable.
- E4: Multiple expiration dates in a bulk package → motivates batch-level tracking if expiration awareness is important.
- E5: Expiration date unreadable/missing → household cannot reliably judge expiration status.
- E6: Membership change (move-out) → household needs to update who participates and who is informed.

3.1.1.9. Ownership structures for shared inventory items

The rough sketch suggests that household items do not all behave the same way in shared environments. Some are treated as clearly shared essentials, while others are managed more personally even within the same household. For that reason, ownership must be analyzed as a domain concept rather than assumed to have only one structure.

Ownership structures are evaluated here based on their ability to:

- support collaborative interaction within a shared household context
- preserve clear responsibility attribution where needed
- align with domain invariants governing households, items, and membership
- support enforceable permission logic
- represent domain relationships coherently without unnecessary complexity

User-owned structure

In a user-owned structure, an item belongs primarily to an individual participant. Shared use is possible, but it must be added on top of individual ownership.

Relationship idea:

Item → User
Item associated with Household context

This structure provides strong individual accountability and clear ownership-based authority, but it does not fit shared-household semantics naturally. Collaboration becomes secondary, because the item is conceptually owned first by a person rather than by the shared household context.

This structure is most suitable when items are primarily personal contributions that only occasionally appear in a shared environment.

Group-owned structure

In a group-owned structure, an item belongs to the shared household context rather than to a specific person. Household members interact with the item through membership.

Relationship idea:

Household → Item
Household member → Membership → Household

This structure supports shared use naturally and aligns well with communal resources. However, it does not provide inherent responsibility attribution. If the domain needs to represent who is primarily responsible for a shared item, that information must be added separately.

This structure is most suitable when items are treated as collective household resources.

Hybrid structure

In a hybrid structure, an item exists within the shared household context but may also be associated with a responsible household member when needed.

Relationship idea:

Household → Item
Household member → Membership → Household
Membership or responsibility relation → Item

This structure preserves the shared household context while still allowing responsibility attribution. It supports both collaboration and accountability, though it is conceptually more complex than the other two structures.

This structure is most suitable when the domain must represent both shared items and items with a clearer responsible participant.

Comparative evaluation

Structure	Collaboration Support	Accountability	Alignment with Domain Invariants	Conceptual Fit
User-Owned	Indirect	Strong	Partial	Moderate
Group-Owned	Native	Weak	Strong	Strong
Hybrid	Native	Strong	Strong	Strong

Recommendation

The hybrid structure provides the strongest overall conceptual fit for the current domain model.

It should be preferred because it:

- preserves the shared household context
- supports collaborative interaction naturally
- allows responsibility attribution when needed
- aligns with membership-based participation
- provides a flexible basis for later permission and ownership rules

This recommendation does not require every item to behave the same way. Instead, it provides a conceptual basis for representing both shared and personal responsibility inside one coherent household model.

3.1.2. Ownership Implications for Permissions and Responsibility Attribution

3.1.2.1. Objective

This document analyzes how item ownership influences permissions and responsibility attribution within a shared inventory system. The goal is to ensure that the selected ownership structure supports enforceable control, accountability, and consistent system behavior.

This analysis builds upon previously defined ownership models, domain invariants, and the selected hybrid ownership structure.

3.1.2.2. Ownership as a Behavioral Concept

Ownership is not only a structural relationship but also a behavioral one. It defines authority over resources and identifies responsibility within a shared context.

In a collaborative inventory domain, ownership must support:

- permission determination
- responsibility attribution
- consistent interaction within shared environments

3.1.2.3. Relationship Between Ownership and Membership

Ownership exists within an inventory context and is scoped by membership. Users interact with items through their participation in the shared inventory.

Conceptual interaction structure:

User → Membership → Inventory → Item Membership → Responsibility for Item

Implications:

- ownership requires membership
- permissions are constrained by shared context

- responsibility is attributed within group participation

3.1.2.4. Ownership and Permission Control

Ownership contributes to permission determination by identifying the participant with primary authority over an item.

Permission sources include:

- membership roles defining baseline capabilities
- ownership responsibility defining elevated authority

Conceptual permission structure:

Responsible Member	Inventory Member	Inventory Administrators
• may manage item state and properties • holds primary decision authority	• interact according to membership role • do not possess inherent ownership authority	• may manage ownership assignments • may override ownership constraints

Implications:

- ownership refines authority within shared access
- permission logic depends on both membership and ownership relationships
- authority is scoped to shared context

3.1.2.5. Ownership and Responsibility Attribution

Ownership provides a mechanism for associating responsibility with shared resources.

Conceptual responsibility model:

- items exist within shared context
- ownership identifies responsible participant
- actions involving items can be attributed through ownership

Implications:

- accountability is preserved in collaborative environments
- responsibility is never undefined
- ownership supports attribution of actions or resource-related outcomes

3.1.2.6. Conceptual Requirements for Ownership Structure

To support permission control and responsibility attribution, the ownership model must ensure:

- ownership exists within an inventory context

- only members may hold ownership responsibility
- items always have a defined responsibility structure
- permissions can be derived from ownership and membership relationships
- responsibility attribution is structurally supported

3.1.2.7. Implications for Conceptual Schema Design

This analysis establishes behavioral constraints that the conceptual schema must support:

- items must reference an inventory context
- membership relationships must connect users to shared resources
- ownership responsibility must be representable within membership scope
- permission and responsibility relationships must be derivable from conceptual structure

These findings validate the hybrid ownership model as a coherent foundation for defining item ownership rules and shared inventory interaction behavior.

3.1.2.8. Domain Attributes Classification for Core Inventory

This section applies the lecture topic Domain Attributes to the Core Inventory subsystem by classifying key domain entities and behaviors according to their attributes (Static, Dynamic, Dynamic Inert, Dynamic Active, Discrete).

1. Item Type

Represents an item definition (e.g., "Clorox Bleach 64oz Original").

Classification: *Static *Discrete

An item type's definition does not change over time once created. Its properties (name, barcode, brand, size) remain constant — a "Clorox Bleach 64oz Original" will remain the same regardless of the passage of time, thus it is classified as Static. It is also Discrete because each item type is its own distinct entity in the system.

2. Item Instance (Package)

Represents a specific purchase or concrete unit of an item type, including information such as date of purchase, storage location, and date of expiration if applicable.

Classification: *Dynamic Inert *Discrete

An item instance changes only due to external actions such as household member updates, consumption, or location change. Since it is changed via external stimuli from the household member it is considered Dynamic Inert. Each item instance has a separate record in the system, making it Discrete.

3. Household

Represents a living unit that groups locations and inventory items.

Classification: *Static *Discrete

A household is a structural entity within the system and does not change state over time since it serves as a container for other entities. Each household is unique making it Discrete.

4. Location

Represents a storage place within a household (e.g., kitchen cabinet, pantry, fridge, bathroom closet).

Classification: *Static *Discrete

Locations are structural subdivisions of a household and do not change autonomously, meaning that they are Static entities. Each location is uniquely represented, so it is Discrete.

5. Household Member

Represents a person who belongs to a household and participates in consuming, replenishing, or coordinating around items.

Classification: *Static *Discrete

A household member's identity does not change over time; while item associations may change, the household member entity itself remains stable so it is Static. Each household member is their own distinct entity, making it Discrete.

6. Duplicate Detection Logic

System logic that determines whether a new entry is a duplicate at either the item type or the item instance level.

Classification: *Dynamic Active

Duplicate detection executes a rule-based behavior when household members add or scan items. It responds to system events and applies programmed rule conditions. Therefore, the Duplicate Detection Logic is classified as Dynamic Active since it can change via internally programmed rules and various conditions.

Evidence Concept Analysis

This subsection documents how raw observations from the domain rough sketch were transformed into structured domain concepts. The goal is to ensure traceability between the initial unstructured notes and the final abstractions used in the domain model.

Rough Sketch vs Concept Formation

Rough Sketch Statement	Derived Concept
"All participants said they have at some point forgotten what they already had at home." (Brainstorming session)	Household Item

Rough Sketch Statement	Derived Concept
"Milk is usually bought twice in the same week at least once per month." (Shared apartment interview)	Duplicate purchase / Coordination failure
"Yogurts sometimes expire; estimated 3–5 thrown away last semester." (Solo student interview)	Expiration Date
"when we open the backup." / "We have rice in two places." (Shared apartment interview)	Backup supply (multiple instances of a Household Item)
"toilet paper runs out unexpectedly when guests visit." (Household owner interview)	Availability (qualitative state: full, almost empty, backup) / Low-stock judgment

By analyzing these statements side-by-side:

- Different expressions referring to "groceries", "items", and "products" are unified under the concept of **Household Item** . - Repeated mentions of duplicates and shortages indicate coordination issues within shared households. - Time-based concerns introduce the concept of **Expiration Date** tied to specific item instances.

Concept Formation Decisions

The following abstractions were introduced by grouping similar ideas:

- "Groceries", "items", "products" - **Household Item** - Multiple physical units - **Item Instance** - "Extra one" - **Backup supply** - "Expires" - **Expiration Date** - "Running out" - **Availability** (with qualitative judgments: full, almost empty, backup)

These groupings allow consistent reasoning across different user statements and scenarios.

Identifying Concept Relationships

From the analysis:

- A **Household Item** may have multiple **Item Instances** - Each **Item Instance** may have an **Expiration Date** - A **Household Item** can be associated with an **Availability** state (qualitative assessment: full, almost empty, backup) - Household members interact with shared items in a common environment

Terminology Alignment

Potential inconsistencies were identified and resolved:

- "Item", "product", and "groceries" - standardized as **Household Item** - "Running out" vs "not available" - unified under **Availability** - "Extra" vs "backup" - unified as **Backup supply**

Domain Scope Note

This analysis focuses strictly on domain concepts and avoids system-specific terminology. Instead of

referring to "users", domain roles such as household members, roommates, or students are implied.

This section highlights a representative subset of concept formation decisions. The parent concept analysis derives approximately 15 distinct concepts; this section traces 5 foundational concepts to demonstrate the methodology of mapping raw observations to domain abstractions.

3.1.2.9. Trace to earlier documentation

This domain concept model is grounded in the domain rough sketch's concrete examples of informal tracking, multiple storage locations, qualitative low-stock judgments, bulk buying, expiration waste, and shared-household coordination failures. It supports the needs described in the informative part (minimizing effort while maintaining reliable availability understanding, and improving shared-household coordination) without committing to specific implementation decisions.

3.1.3. Requirements Concept Analysis

This section analyzes requirement-level concepts derived from the domain rough sketch and relates them to the domain concepts established earlier in the document. Its purpose is to define the vocabulary and tradeoffs that later requirements must use consistently.

3.1.3.1. Starting Point: Requirement Drivers Anchored in the Rough Sketch

The rough sketch motivates the following requirement drivers:

- **Low-friction capture:** "If it takes more than a few seconds to log, I won't do it."
- **Shared coordination failures:** duplicate purchases, no clear replenishment responsibility, and reliance on fridge notes or chat.
- **Multiple storage locations:** the same item type may exist in more than one place.
- **Qualitative stock judgments:** households often use expressions such as "full", "almost empty", or "backup" instead of exact counts.
- **Expiration and waste:** food and medicine may expire unnoticed, and some items may exist in unopened duplicates.
- **Variable consumption:** exam weeks and guests can change usage rates.
- **Membership change:** households need a clear way to handle move-out and access removal.

These drivers shape the requirement-level concepts and tradeoffs below.

3.1.3.2. Actors and Goals

Actor (domain role)	Primary goals (outcomes)
Student living independently (solo)	Maintain a reliable understanding of what is available with minimal effort; avoid duplicate purchases; avoid unexpected shortages; reduce waste from expired items.

Actor (domain role)	Primary goals (outcomes)
Student living with roommates/cohabitants	Maintain a shared understanding of availability; coordinate replenishment to avoid duplicate or missed purchases; distinguish shared vs personal items without conflict.
Household member with budget-awareness focus	Understand spending patterns for essentials; avoid overspending; plan purchases using summaries rather than memory.
Household organizer	Reduce confusion about where items are stored and how they are grouped; keep organization stable enough that others can follow it.
Operations / monitoring stakeholder	Ensure automated checks and alert generation are reliable, observable, and do not produce duplicates or noisy output.

3.1.3.3. Requirement-Level Concepts

The following concepts are used to express later requirements without prematurely committing to a specific implementation.

Concept	Meaning in requirements
Fast capture	A tracking or update interaction that fits within a few seconds and does not require perfect detail at entry time.
Tracked record vs reality	The system represents availability, but real household inventory changes continuously. Divergence is expected and must be managed.
Shared visibility	Household members can converge on the same understanding of availability and recent changes.
Actionable states	States that directly support decisions, such as `LOW_STOCK`, `OUT_OF_STOCK`, `NEAR_EXPIRY`, and `EXPIRED`.
State transition	A change from one actionable state to another. Transitions are the main trigger for alerts in order to reduce repetition.
Low-stock strategy	The rule by which "low" is determined, such as a fixed threshold or reorder point.
Near-expiry window	The definition of "expiring soon", expressed as a time window before expiration.
Scheduled sweep	A periodic background check that evaluates low-stock and expiration conditions without requiring the household member to open the system.
Notification record	A persisted representation of an alert, including type, time, and read/unread status.
Deduplication / anti-spam	Rules that prevent repeated alerts or repeated list additions when nothing meaningful changed.

Concept	Meaning in requirements
Auto-add to restock list	Rule-based conversion of actionable states into restock-list entries.
Transaction history	An append-only log of quantity changes with reason and timestamp, allowing the current tracked state to be explained.

3.1.3.4. Conflicts and Tradeoffs

The requirement-level concepts above introduce several tensions that later requirements must resolve explicitly:

- **Speed vs accuracy** — requiring too much detail slows capture, but requiring too little may reduce trust in the tracked record.
- **Flexibility vs consistency** — household-specific practices are realistic, but too much freedom increases terminology drift and inconsistent usage.
- **Shared visibility vs privacy** — shared coordination improves when more information is visible, but some items and spending records remain personal.
- **Timely reminders vs notification fatigue** — reminders reduce shortages and waste, but repeated alerts reduce trust and may be disabled.
- **Simplicity vs "smartness"** — fixed thresholds and explainable triggers are robust, while predictive or highly adaptive behavior is harder to justify and verify.
- **Automation reliability vs user-device dependence** — background checks improve consistency, but they require clear observability, repeatability, and error handling.

These tradeoffs explain why later requirements must emphasize low-friction entry, explainable alerting, deduplication, shared-versus-personal separation, and traceable quantity history.

3.1.3.5. AHP-Based Requirement Prioritization

This section documents an AHP-based method for requirement prioritization and provides a reproducible calculation path for the team.

Evaluation Criteria

The prioritization compares requirements using the following criteria:

* **User value** — how strongly the requirement supports waste reduction, coordination, or low-friction capture. * **Feasibility** — how practical the requirement is under the current architecture and project constraints. * **Impact** — how much the requirement improves overall household inventory behavior. * **Implementation complexity** — how difficult the requirement is to build; lower complexity is preferred when value is similar.

Team Input Aggregation Method

Each team member supplies pairwise comparisons for the criteria and for the requirements under each criterion. The shared matrices are produced by averaging the corresponding matrix entries across team members.

```
aggregate_matrix <- function(matrices) {
  Reduce("+", matrices) / length(matrices)
}
```

AHP Process Steps

. Collect pairwise comparisons from all team members for the evaluation criteria. . Aggregate the criteria matrices. . Collect pairwise comparisons for the requirements under each criterion. . Aggregate the requirement matrices for each criterion. . Compute criterion weights and local requirement weights. . Compute global scores from local weights and criterion weights. . Rank requirements by final score.

Sample Aggregated AHP Result

The script `docs/3-analytic/concept-analysis/requirements-concept-analysis/ahp-prioritization.R` demonstrates the method and can be updated with real team input.

Rank	Requirement
1	Core inventory capture and low-friction item entry
2	Shared visibility and household coordination
3	Low-stock and out-of-stock awareness
4	Expiration awareness and alerting
5	Restock list auto-add behavior
6	Transaction history and traceability

The AHP method makes the prioritization process transparent, repeatable, and easier to defend during review.

3.2. Domain Analysis and Concept Formation Pipeline

3.2.1. Overview

This document describes the structured pipeline used to process raw domain description units into higher-level abstractions for the Home Inventory Management System. The pipeline moves from unstructured observations and low-level entity lists toward a coherent set of named domain concepts and a unified narrative, while surfacing inconsistencies and gaps for analyst review.

The pipeline consists of four sequential stages:

```
[Stage 1: Ingest]
   Raw Domain Description Units
         ▯
[Stage 2: Concept Formation]
   Group low-level entities → named domain concepts
         ▯
```

```
[Stage 3: Narrative Rewrite]
   Replace entity lists with concept terminology
         ▯
[Stage 4: Inconsistency Detection]
   Flag P / not-P conflicts, gaps, and incompletenesses
         ▯
[Output: Analyst Review Package]
   Unified narrative + flagged issues
```

3.2.2. Stage 1 — Ingest: Raw Domain Description Units

Raw domain description units (DDUs) are the unprocessed observations, interviews, and artifacts collected during domain acquisition. For this project the raw DDUs were gathered from student surveys, informal household practices, and course domain analysis exercises.

3.2.2.1. Collected DDUs

The following table lists every raw DDU ingested, organized by source section:

DDU ID	Source Section	Raw Description
DDU-01	Narrative	Students buy food and supplies irregularly, store them in household locations, and consume them gradually.
DDU-02	Narrative	When items run low or out, someone must notice and replenish.
DDU-03	Narrative	Tracked understanding diverges from reality; relies on memory, fridge notes, chat messages, pantry checks.
DDU-04	Narrative	Items may be forgotten in different locations; backups opened without recording.
DDU-05	Narrative	Duplicates purchased due to incomplete information.
DDU-06	Narrative	Exam weeks and guest visits increase usage rates; variable consumption patterns.
DDU-07	Narrative	Bulk purchases create multiple items with different expiration dates.
DDU-08	Narrative	Spending influences decisions — bulk for lower price, delay restocks, cheaper alternatives.
DDU-09	Narrative	In shared households, disputes arise over personal vs. shared purchases.
DDU-10	Entities	pantry items, bathroom products, cleaning supplies, medicine, utilities, kitchen items.

DDU ID	Source Section	Raw Description
DDU-11	Entities	kitchen, bathroom, bedroom, pantry, storage closet, hallway shelf.
DDU-12	Entities	purchase date, expiration date, quantity, price, notes.
DDU-13	Entities	running low, out of stock, near expiry, expired.
DDU-14	Entities	restock, shopping trip, grocery run, replenishment event.
DDU-15	Entities	budget tracking, spending history, spending category.
DDU-16	Entities	notification, alert, reminder, push notification.
DDU-17	Entities	household, student, roommate, household manager, household member.
DDU-18	Behaviors	Household maintenance cycle: acquire → store → consume → judge → replenish.
DDU-19	Behaviors	Shared-household coordination: shared vs. personal item scoping, duplicate prevention.
DDU-20	Functions	Low-stock detection: threshold logic, qualitative vs. quantitative judgments.
DDU-21	Functions	Budget management: cycle-based limits, threshold crossing alerts, spending attribution.
DDU-22	Artifacts	Fridge notes, whiteboards, group chat messages, notes apps, pantry photos, receipts.

3.2.3. Stage 2 — Concept Formation: Abstraction Mapping

In this stage, low-level entity lists from the raw DDUs are grouped into named domain concepts at a higher abstraction level. The goal is to identify the **kinds of things** the domain talks about, independent of any specific implementation.

3.2.3.1. Concept Abstraction Table

Domain Concept	Raw DDU Entities Grouped Under This Concept	Source DDUs
Household Item	pantry items, bathroom products, cleaning supplies, medicine, utilities, kitchen items, food, supplies	DDU-01, DDU-10
Location	kitchen, bathroom, bedroom, pantry, storage closet, hallway shelf, household locations	DDU-01, DDU-11

Domain Concept	Raw DDU Entities Grouped Under This Concept	Source DDUs
Item Attributes	purchase date, expiration date, quantity, price, notes, batch, variant, size, brand	DDU-07, DDU-12
Availability State	running low, out of stock, near expiry, expired, "feels light", "opened backup", qualitative judgment	DDU-02, DDU-04, DDU-13, DDU-20
Tracked Inventory Record	fridge notes, chat messages, pantry checks, notes app lists, pantry photos, memory-based estimates	DDU-03, DDU-22
Replenishment Action	restock, shopping trip, grocery run, replenishment event, acquiring more	DDU-02, DDU-14
Expenditure	spending history, spending category, receipt, price paid, shared purchase, personal purchase	DDU-08, DDU-09, DDU-15
Budget	budget tracking, cycle-based limits, threshold, budget status, spending cap	DDU-08, DDU-15, DDU-21
Alert / Notification	notification, alert, reminder, push notification, low-stock alert, expiration alert, budget alert	DDU-16
Household	household, living unit, shared home, student household	DDU-17, DDU-18
Household Member	student, roommate, household manager, participant, member, occupant	DDU-17, DDU-19
Coordination Failure	duplicate purchase, missing update, stale information, ownership dispute, inconsistent records	DDU-04, DDU-05, DDU-09, DDU-19

3.2.3.2. Concept Hierarchy

The following hierarchy shows how concepts relate to one another:

```
Household
 ├─── Household Member (1 or more)
 └─── Inventory (reality)
      └─── Household Item (many)
           ├─── Item Attributes (expiration, quantity, price...)
           ├─── Location (where stored)
           └─── Availability State (OK / Low / Out / Expired)

Tracked Inventory Record  ← approximates → Inventory (reality)

Replenishment Action  ← triggered by → Availability State (Low / Out)
Expenditure          ← recorded after → Replenishment Action
```

```
Budget                ← governs → Expenditure

Alert / Notification ← triggered by → Availability State or Budget threshold
Coordination Failure ← arises from → Tracked Inventory Record diverging from reality
```

3.2.4. Stage 3 — Narrative Rewrite

See: Unified Domain Narrative

The unified narrative replaces all raw entity lists (DDU-10 through DDU-22) with their corresponding concept names from Stage 2. All terminology is drawn exclusively from the project's Ubiquitous Language dictionary (`docs/2-descriptive/domain-description/terminology/terminology.adoc`).

3.2.5. Stage 4 — Inconsistency Detection

See: Inconsistency Detection Report

The inconsistency detector scans all DDUs and existing domain documents for:

- **Direct P / not-P conflicts** — one unit asserts a property that another directly negates.
- **Scope conflicts** — a rule applies a constraint that another rule exempts.
- **Completeness gaps** — a concept is referenced but never fully defined.
- **Terminology drift** — the same real-world phenomenon described under two different names.

3.2.6. Output: Analyst Review Package

The full analyst review package produced by this pipeline consists of:

Artifact	Location
Concept Abstraction Mapping	This document — Stage 2 table above
Unified Domain Narrative	unified-domain-narrative.adoc
Inconsistency Detection Report	inconsistency-detection.adoc
Existing Terminology Dictionary	`docs/2-descriptive/domain-description/terminology/terminology.adoc`
Existing Domain Concept Analysis	`docs/3-analytic/concept-analysis/domain-concept-analysis/domain-concept-analysis.adoc`

3.3. Requirements Inconsistency Analysis

3.3.1. Purpose

Systematically identify, categorize, and resolve inconsistencies within the current system requirements. This includes detecting clashes such as terminology inconsistencies, timing conflicts, overlapping features, or contradictions between functional and non-functional requirements. This is specific to section 3.1 of the documentation.

3.3.2. Terminology Inconsistencies

3.3.2.1. Documents Containing Issues

- automation-check-system.adoc - definition-expiration-alert-system.adoc - evaluationOfNotification.adoc - grocerylist_autoadd_decisionlogic.adoc - inventory-transaction-tracking.adoc - low-stock-research-strategies.adoc - notifications_supabase_schema.adoc - requirements-concept-analysis.adoc - requirements-concept-analysis(fernan).adoc - requirements-evidence-concept-analysis.adoc - schema-for-inventory-&-stock-changes.adoc

3.3.2.2. Problem

Different documents use different terminology to refer to similar concepts:Z - Item instead of item type or item instance (package) - Stock change instead of transaction (quantity change) - Inventory item instead of item type or item instance (package) - Stock item instead of tracked inventory record, item type, or item instance (package), depending on context - Product instead of item type - Grocery item instead of item type - Grocery list instead of restock list / planner - User instead of household member Notification record/event instead of alert / notification record

This creates ambiguity about whether these represent different entities or the same entity in different contexts. While some terms are used interchangeably, it is unclear if they are meant to be distinct, so it is better to standardize terminology in all sections.

3.3.2.3. Resolution

Revise the official terminology list in `terminology.adoc` to ensure consistent use of terms across all documents. Make changes needed within the documents so they align with the standardized terminology. For example: in all documents pertaining to Section 3.1, change all instances of "user" to "household member".

3.3.2.4. Final Decision

All documents in Section 3.1 must use the standardized terminology defined in `terminology.adoc`. Synonyms such as "inventory item", "stock change", "product", "grocery item", and "user" will be replaced with the official terms: - tracked inventory record - transaction - item type - item instance - household member - restock list / planner (terms may be used interchangeably depending on context) - alert / notification record (terms may be used interchangeably depending on context)

Terminology definitions will exist only in `terminology.adoc`, and other documents must use the terms without redefining them.

3.3.3. Timing Conflicts

3.3.3.1. Documents Containing Issues

- automation-check-system.adoc - definition-expiration-alert-system.adoc - evaluationOfNotification.adoc - low-stock-research-strategies.adoc

3.3.3.2. Problem

Some documents describe: - Real-time detection - Scheduled daily sweeps - Cron jobs - Polling mechanisms

This creates ambiguity about when alerts should be generated and when automation logic should run.

3.3.3.3. Resolution

Take the already defined rules and ensure consistency across all documents. Make any additions or modifications needed to clarify when sweeps happen and when alerts are generated. For example, in `definition-expiration-alert-system.adoc`, clarify that expiration and near-expiry conditions are evaluated during the scheduled sweep, and that alerts are only generated when a state transition occurs.

3.3.3.4. Final Decision

The system will follow a hybrid timing model:

- Immediate evaluation occurs after each transaction to detect low-stock and out-of-stock state transitions. - A scheduled sweep runs once per day to evaluate expiration, near-expiry, and low-stock conditions for items that have not recently changed. - Alert / notification records are generated only when a state transition is detected, not continuously while a condition remains true.

This timing model must be reflected consistently across all automation, expiration, and notification documents.

3.3.4. Overlapping Feature Responsibilities

3.3.4.1. Documents Containing Issues

- low-stock-research-strategies.adoc - grocerylist_autoadd_decisionlogic.adoc - automation-check-system.adoc - definition-expiration-alert-system.adoc

3.3.4.2. Problem

Low stock detection, expiration alerts, grocery auto-add, and notifications overlap in responsibilities. It is unclear which subsystem is responsible for triggering each action.

For example: - Low stock triggers alerts - Low stock triggers restock list auto-add - Expired items trigger notifications - Expired items may also trigger restock list auto-add

This creates feature overlap and unclear system boundaries.

3.3.4.3. Resolution

In each feature document, clearly define the responsibility boundary for that feature. For example, in `definition-expiration-alert-system.adoc`, clarify that expiration-related state detection belongs to the Automation System, while creation of alert / notification records belongs to the Notification Service.

3.3.4.4. Final Decision

System responsibilities are separated as follows:

- Transactions are responsible only for recording quantity changes and updating tracked inventory records. - The Automation System is responsible for evaluating low-stock, near-expiry, expired, and out-of-stock conditions and detecting state transitions. - The Notification Service is responsible for creating alert / notification records. - The Restock List / Planner system is responsible for adding item types to the restock list when specific state transitions occur.

Each subsystem must only perform its defined responsibility to avoid feature overlap.

3.3.5. Data Model vs Transaction Model Conflict

3.3.5.1. Documents Containing Issues

- schema-for-inventory-&-stock-changes.adoc - inventory-transaction-tracking.adoc

3.3.5.2. Problem

The system includes: - tracked inventory records that store current quantity - transaction (quantity change) records that track quantity history

It is unclear whether: - current quantity is stored directly in the tracked inventory record, or - current quantity is derived by applying all transaction (quantity change) records

This creates ambiguity in the data model and may lead to inconsistent implementation, testing, and automation behavior.

3.3.5.3. Resolution

Define the following rule:

Tracked inventory records store the current quantity and are updated whenever a transaction (quantity change) is recorded.

Transaction (quantity change) records serve as append-only historical records that explain how the current quantity was reached over time. They are not the primary source used to compute current quantity during normal operation.

3.3.5.4. Final Decision

Tracked inventory records will store the current quantity directly. Transactions are append-only historical records that explain how the current quantity was reached. When a transaction is

recorded, the tracked inventory record quantity is updated immediately.

Current quantity will not be computed by replaying all transactions during normal operation. Transactions are used for history, auditing, and analytics, not for real-time quantity calculation.

3.3.6. Notification System Responsibility Conflict

3.3.6.1. Documents Containing Issues

- notifications_supabase_schema.adoc - automation-check-system.adoc - evaluationOfNotification.adoc - definition-expiration-alert-system.adoc

3.3.6.2. Problem

It is unclear which subsystem is responsible for creating alert / notification records after low-stock, near-expiry, expired, or out-of-stock conditions are detected.

Different documents can be interpreted as assigning this responsibility to: - the Automation System - database triggers - the expiration or low-stock logic - a Notification Service

This creates unclear system responsibility boundaries.

3.3.6.3. Resolution

In all relevant documents, clarify that the Automation System is responsible for evaluating low-stock, near-expiry, and expired conditions and detecting state transitions. When a new relevant state transition is detected, the Automation System triggers downstream notification handling. The Automation System does not own notification delivery. Creation of alert / notification records belongs to the Notification Service. In-app notification records are then displayed through the application interface. Push delivery, if enabled later, belongs to the Push Service.

3.3.6.4. Final Decision

The Automation System is responsible for detecting state transitions such as LOW_STOCK, OUT_OF_STOCK, NEAR_EXPIRY, and EXPIRED.

When a relevant state transition is detected, the Automation System triggers the Notification Service.

The Notification Service is responsible for creating alert / notification records. The application interface is responsible for displaying in-app notifications. Push delivery, if implemented later, will be handled by a separate Push Service.

This separation ensures clear system boundaries and prevents overlapping responsibilities.

3.3.6.5. Requirement Concept Overlap

Concept overlaps were fixed while addressing the problems with the other categories. For example, the concept of "state transition" was clarified in the context of both low-stock and expiration detection, which resolves overlaps between those features and their associated alerts and notifications.

3.3.7. Conclusion

Based on the identified problems,the system architecture should follow this flow:

Tracked Inventory Records → Transactions (Quantity Changes) → State Evaluation → Automation → Alert / Notification Records + Restock List / Planner Auto-Add

Defining subsystem responsibilities and terminology will resolve most requirement clashes.

3.3.7.1. Risk Identification using FMEA and Guidewords

This section identifies requirement-level risks using two structured techniques: Failure Mode and Effects Analysis (FMEA) and Guideword analysis.

The purpose of this subsection is to identify realistic failure situations and edge cases related to the home inventory system's core functionality. These risks are requirement-level concerns that may affect reliability, usability, correctness, and trust.

FMEA Risk Case 1 — Incorrect Quantity After Transaction

Field	Description
Failure mode	The quantity in a tracked inventory record becomes incorrect after a transaction is recorded.
Cause	A transaction is recorded with the wrong quantity, multiple household members update the same tracked inventory record close together, or a manual adjustment is entered incorrectly.
Effect on the system	Low-stock judgment may become incorrect, alert / notification records may be triggered unnecessarily or not triggered when needed, and the restock list / planner may contain incorrect item types.
Severity	8
Occurrence	5
Detection	5
Risk Priority Number	200
Scenario	One household member records replenishment while another records consumption for the same tracked inventory record. The final recorded quantity does not reflect the actual household inventory, causing an incorrect low-stock judgment.

FMEA Risk Case 2 — Scheduled Sweep Does Not Run

Field	Description
Failure mode	The scheduled sweep does not execute at the expected time.
Cause	Scheduler failure, missed cron execution, service downtime, or configuration errors in the automation process.

Field	Description
Effect on the system	Near-expiry and expired item instances are not evaluated on time, low-stock conditions may remain undetected, and alert / notification records may be delayed or not created.
Severity	9
Occurrence	3
Detection	4
Risk Priority Number	108
Scenario	The scheduled sweep is expected to run once per day, but an automation failure prevents the run from occurring. Expired item instances remain undetected until a later run or manual review.

FMEA Risk Case 3 — Duplicate Alert / Notification Records

Field	Description
Failure mode	The system creates duplicate alert / notification records for the same unchanged condition.
Cause	State transitions are not checked correctly, deduplication rules are incomplete, or retries reinsert the same alert / notification record.
Effect on the system	Household members may experience notification fatigue, ignore future alerts, or lose trust in the system's automation.
Severity	7
Occurrence	4
Detection	4
Risk Priority Number	112
Scenario	A tracked inventory record remains LOW_STOCK across several evaluations. If the system creates a new alert / notification record each time without checking whether the state changed, duplicate records accumulate.

Guideword Case 1 — Missing

Guideword: Missing

Scenario: A household member consumes or replenishes an item type but does not record the transaction.

System impact: The tracked inventory record diverges from household inventory reality, low-stock judgment becomes less reliable, and later automation decisions may be incorrect.

Possible mitigation: Support low-friction logging, allow quick manual adjustments, and use transaction history to correct tracked inventory records over time.

Guideword Case 2 — Delayed

Guideword: Delayed

Scenario: The scheduled sweep runs later than expected, so near-expiry or expired item instances are detected after the intended time.

System impact: Alert / notification records are generated late, household members may not act in time, and expired item instances may remain in use longer than expected.

Possible mitigation: Record the last successful scheduled sweep run, detect missed runs, and execute a catch-up sweep when necessary.

Guideword Case 3 — Duplicate

Guideword: Duplicate

Scenario: The same item type is repeatedly added to the restock list / planner even though it is already present.

System impact: The restock list / planner becomes cluttered, household members may become confused, and the system may appear unreliable or overly aggressive.

Possible mitigation: Require deduplication before auto-add, allow household members to remove item types manually, and re-add only when a new qualifying state transition occurs.

Relationship to Requirement Concepts

These risks are directly related to the requirement-level concepts already identified in this section:

- **Fast capture** relates to the risk of missing transactions.
- **Tracked record vs reality** relates to quantity drift and inaccurate tracked inventory records.
- **Scheduled sweep** relates to delayed or missing automation runs.
- **State transition** relates to duplicate alert / notification records.
- **Auto-add to restock list / planner** relates to duplicate restock entries and user-control concerns.

This risk analysis supports later design decisions, validation activities, and reliability considerations.

3.4. Validation and Verification

Validation ensures we are building the right thing (meets real needs). Verification ensures we built it correctly (meets requirements/specs).

3.4.1. Scenarios

Scenarios describe realistic end-to-end situations in the domain and how stakeholders expect things to work.

3.4.1.1. Scenario S1a — Low-Stock Trigger After Partial Consumption

* Persona/actor: Julian, the health-conscious student; household member. * Context: Julian keeps a personal shelf of meal-prep items separate from roommates. He depends on Barilla gluten-free elbows, which are not available locally and require online ordering with 3-day shipping. His low-stock threshold for this item is set to 2 boxes. * Trigger: Julian records consumption after Sunday meal prep, reducing his tracked quantity from 3 boxes to 1 box—below his low-stock threshold of 2. * Expected state transition: NORMAL → LOW_STOCK

.Steps . Julian opens the system after Sunday meal prep and selects "Barilla gluten-free elbows" from his personal inventory shelf. . In the app Julian sees that 3 boxes had remained before cooking this dinner. He used 2 boxes for the week's meal prep and decreases the count to 1. . The system compares the new count to the low-stock threshold and flags Barilla gluten-free elbows as low-stock. . The system adds "Barilla gluten-free elbows" to the restock list/planner with status "Needs restock" and reason "LOW_STOCK". . The system records and presents a low-stock alert to Julian: "Barilla gluten-free elbows is low — 1 box remaining."

.Expected outcome * The inventory shows "Barilla gluten-free elbows: 1 box" with a LOW_STOCK flag. * The item appears in the restock list/planner as needing replenishment. * Julian has enough lead time to reorder before running out mid-week.

3.4.1.2. Scenario S1b — Low-Stock and Out-of-Stock Simultaneously

* Persona/actor: Julian, the health-conscious student; household member. * Context: Same as S1a, but Julian had only 2 boxes of Barilla gluten-free elbows remaining before preparing dinner. His low-stock threshold is set to 2 boxes. * Trigger: Julian uses both remaining boxes for meal prep and decreases the count from 2 to 0. * Expected state transition: NORMAL → OUT_OF_STOCK

.Steps . Julian opens the system after meal prep and selects "Barilla gluten-free elbows." . He had 2 boxes remaining. He used both and decreases the count to 0. . The system detects that the new count is both below the low-stock threshold and at zero. The system flags the item as OUT_OF_STOCK. . The system creates a single combined alert: "Barilla gluten-free elbows is out of stock." The out-of-stock condition takes priority over low-stock to avoid redundant alerts. . The restock list/planner entry is created with reason "OUT_OF_STOCK."

.Expected outcome * Julian receives one alert, not two separate alerts. * The restock list/planner shows the item with the most urgent reason. * Julian's entire meal plan for the week is at risk; the alert conveys urgency.

.Related needs / stories * Needs: N1, N2 * User stories: **US-ALT-01** US-INV-01

3.4.1.3. Scenario S2 — Shared Inventory Prevents Duplicate Purchase

* Persona/actor: Sofia, the budget-conscious roommate; Andrea, the unofficial household manager; household members. * Context: Sofia and her roommates share household essentials. Past issues include duplicate purchases due to out-of-date whiteboards and unclear responsibility for shared items. Sofia has a strict $50 weekly grocery budget. * Trigger: Sofia is at the supermarket and is about to buy Charmin Ultra Soft toilet paper. She checks the shared inventory before paying. * Expected state transition: No state change for "Charmin Ultra Soft toilet paper"; shared visibility converges after synchronization

.Steps . Sofia opens the shared inventory on her phone at the supermarket and searches for "toilet paper." . The system displays: "Charmin Ultra Soft toilet paper — In stock: 24 rolls." . Sofia sees the item is sufficiently stocked and puts the 12-pack back on the shelf, saving $8.99 of her $50 weekly budget. . Later that afternoon, Andrea records a purchase of Dawn dish soap at the wholesale club. The shared inventory updates for all household members once the change is recorded and synchronized.

.Expected outcome * Sofia avoids buying duplicate toilet paper, keeping $8.99 within her budget. * All household members see the updated shared inventory after synchronization.

.Related needs / stories * Needs: N3, N4, N5 * User stories: **US-INV-02** US-SB-01 ** US-RPT-01

3.4.2. Walkthroughs

Walkthroughs are a validation activity: they check whether the team's understanding of the domain and scenarios matches stakeholder expectations. They do not verify implementation correctness.

3.4.2.1. Walkthrough plan

- Participants:
  - Development team members.
  - 2–3 prospective users acting as domain informants.
- Materials:
  - Event table and Important Event Sequences from the domain description.
  - Scenario descriptions
  - Draft UI mockups or simple screen sketches.
  - Terminology list defining key concepts such as household, inventory, item type/item instance, restock list/planner, and spending tracking.
  - Expiration state model summary from `alert-system-states.adoc`.
  - Technical delivery considerations for expiration alerts from `tech-expiration-alert-system.adoc` (used only to sanity-check feasibility; not evaluated with stakeholders).
- Method:
  - Scenario-driven walkthrough using a think-aloud approach (participants verbalize what they expect and why at each step).
  - Facilitator presents a situation and asks participants to describe what they expect to happen next.
  - Event sequence is compared live against participant expectations and domain terminology.
  - Clarifications and disagreements are documented as validation findings.
  - No implementation details are discussed. The focus remains on domain correctness and stakeholder mental models.

3.4.2.2. Walkthrough questions

- What part of this process feels different from how you normally manage items at home?
- Does this sequence of events reflect what actually happens when something runs out or runs low?
- At this point in the scenario, what would you expect the system-to-be to do next?
- Are any terms unclear or unnatural?
- Would you ever ignore or bypass this step in real life? Why?
- Is there information you would need that is not represented here?
- Is this the order in which these things would normally occur?
- When would this feature become annoying or unnecessary?

These questions aim to validate mental models and domain assumptions, not measure satisfaction.

3.4.2.3. Hypotheses to validate

Hypothesis	What to observe / ask	Expected impact if confirmed
Low-stock should not trigger immediately after every quantity decrease.	Do participants expect an alert only when quantity crosses a chosen threshold? Record quotes with participant code and date.	Clarify low-stock meaning in terminology; specify threshold-based triggering and deduplication rules.
Expiration does not always imply unavailability.	Do participants treat expiration differently by item type? Record evidence with participant code and date.	Separate “expired” from “unavailable” states; require a policy for what happens on expiration.
Restock list/planner should allow manual override.	Do participants want all low/expired items auto-added, or do they prefer control? Record evidence with participant code and date.	Specify rules for manual remove/defer and for auto-add gating to reduce clutter.
Shared-household alerts may be excessive.	Do multiple household members receiving redundant alerts perceive them as noisy? Record evidence with participant code and date.	Add preferences or scoping; emphasize grouping/deduplication and “transition-only” alerting.

3.4.3. Testing

Testing is the main verification mechanism for this project. For Milestone 1, this section defines the verification approach, the main test types, and an initial set of representative test cases.

3.4.3.1. Verification Strategy

- **Code review:** all changes to `main` must go through a pull request reviewed by at least one teammate who did not author the change. Reviews check correctness against requirements, edge cases, terminology consistency, and whether tests were added or updated.
- **Automated tests:** unit, integration, and verification tests run on pull requests and before merge. At minimum, affected tests must pass before changes are merged.
- **Manual checks:** short repeatable smoke checks are performed after merges for stakeholder-visible behaviors such as item updates, alerts, and shared inventory consistency.

3.4.3.2. Test Types

- **Unit tests:** verify isolated rules such as threshold crossing, expiration handling, ownership changes, state transitions, and transaction creation.
- **Integration tests:** verify persistence, retrieval, synchronization, and automation behavior across connected components.
- **End-to-end tests:** verify complete workflows such as adding/editing items, triggering low-stock conditions, and generating alerts without duplication.
- **UI tests:** verify that key screens present the correct state and controls for milestone features.

3.4.3.3. State Model Coverage

Verification uses the same alert and state vocabulary defined elsewhere in the project:

- Stock-related states: `NORMAL` / `IN_STOCK`, `LOW_STOCK`, `OUT_OF_STOCK`
- Expiration-related states: `Active`, `Expiring`, `Expired`, `Renewed`, `Deleted`

Verification focuses on state transitions and deduplication behavior regardless of delivery mechanism.

3.4.3.4. Initial Test Cases

Test case	What it verifies	Related requirement/story
TC-INV-01	Add Item entry point exists on the main screen.	US-INV-01
TC-INV-02	Saved item appears immediately in inventory without manual refresh.	US-INV-01
TC-INV-03	Missing or unreadable expiration dates are treated as unknown; valid dates persist correctly.	US-INV-01
TC-INV-04	Editing an item updates stored metadata and dependent values.	US-INV-01
TC-INV-05	Each item shows its ownership indicator.	US-INV-02

Test case	What it verifies	Related requirement/story
TC-INV-06	Ownership and purchase information appear in item details when available.	US-INV-02
TC-INV-07	Changing an item from personal to shared propagates consistently after synchronization.	US-INV-02
TC-ALT-01	Low-stock state appears when quantity crosses the threshold.	US-ALT-01
TC-ALT-02	Custom thresholds persist and are used in later checks.	US-ALT-01
TC-ALT-03	Repeated low-stock conditions do not generate spammy duplicate alerts.	US-ALT-01
TC-AUTO-01	Scheduled sweep evaluates threshold and expiration boundaries correctly.	Automation behavior
TC-AUTO-02	Repeated scheduled checks without state change do not create duplicate alerts or list additions.	Automation behavior + deduplication
TC-EXP-01	Expiration states transition correctly from Active to Expiring to Expired.	Expiration behavior
TC-EXP-02	Renewed or deleted items do not keep emitting outdated expiration alerts.	Expiration behavior + deduplication
TC-TXN-01	Every quantity change creates an append-only transaction record with reason and timestamp.	Transaction tracking
TC-SB-01	Shared and personal spending data remain distinct for later summaries and splits.	US-SB-01, US-INV-02
TC-RPT-01	Dashboard aggregates update after synchronized changes.	US-RPT-01

3.4.3.5. Definition of Done

- The feature meets the mapped acceptance criteria for the target user story.
- Relevant automated tests are added or updated and pass before merge.
- Documentation is updated when terminology, rules, or domain behavior changes.
- No unresolved review issues remain for correctness, edge cases, or traceability.

3.4.4. Core Inventory Integrity Concerns Relevant to Verification

This subsection keeps only the integrity concerns that directly affect verification of system behavior. The goal is not to restate a full catalog of runtime field-validation rules, but to identify the constraints that must hold for the tested behaviors to remain meaningful.

The following integrity concerns are relevant to verification:

- quantities must not become negative
- item instances must reference valid item types and valid locations
- expiration dates, when present, must remain logically valid
- quantity-changing actions must create a corresponding transaction record
- shared and personal household data must remain distinguishable where required
- household participation changes must not leave invalid access relationships behind

These concerns matter because incorrect stored data would invalidate later behaviors such as low-stock detection, expiration handling, synchronization, transaction history, and shared-household coordination.

Within this section, these concerns are checked through tests and reviews such as:

- checking that quantity updates do not produce invalid inventory states
- checking that expiration handling behaves correctly when dates are present, missing, or unreadable
- checking that quantity-changing actions preserve append-only transaction history
- checking that synchronization does not break household, location, or membership relationships
- checking that shared and personal purchase data remain correctly distinguishable

Detailed field-level constraints and schema-oriented data rules are documented with the corresponding design and data-model material rather than in this validation and verification section.

3.4.5. Acceptance and End-to-End (E2E) Testing

To provide concrete evidence of our system's reliability, we conducted formal Acceptance and End-to-End (E2E) tests on our minimal viable feature set. These tests validate the user journey from the perspective of our core personas (**Andrea**, **Mateo**, and **Sofia**) and ensure the application handles not only the "happy path" but also boundary conditions and failure states.

3.4.6. Feature 1: Core Inventory & Barcode Scanning

Testing Objective: Verify that a user can seamlessly digitize their physical pantry, and that the system safely handles unrecognized items and missing data.

Test ID	Scenario	Type	Expected Outcome	Actual Outcome	Status
TC-1.1	Scan a globally recognized barcode (e.g., a standard Coca-Cola can).	Happy Path	The API retrieves the item. Name, brand, and image auto-populate. The item saves to Supabase.	Product data populated in 1.2s. Item appeared instantly in list via real-time sync.	**PASS**
TC-1.2	Scan an unrecognized, local store-brand barcode.	Edge Case	System notifies the user item is unrecognized and transitions to Manual Entry fallback UI.	Alert displayed: "Item not found." Transitioned to manual form. User saved custom item.	**PASS**
TC-1.3	Attempt to save an item with a blank expiration date or negative quantity.	Failure State	The UI prevents the transaction and displays inline validation errors.	Save button disabled. Red validation text appeared: "Expiration date is required."	**PASS**

3.4.7. Feature 2: Automated Expiration Alerts & Notifications

Testing Objective: Ensure Supabase Edge Functions correctly identify expiring items and deliver push notifications without spamming the user.

Test ID	Scenario	Type	Expected Outcome	Actual Outcome	Status
TC-2.1	Item expiration date falls within the 3-day warning threshold.	Happy Path	The scheduled Cron job detects the item and delivers a push notification to the device.	Notification received: "Your Milk expires in 3 days!" Item correctly flagged in UI.	**PASS**
TC-2.2	The Cron job runs again the following day for the same item from TC-2.1.	Edge Case	Deduplication logic prevents a second push notification from being sent to avoid fatigue.	System recognized the existing `alert_sent` flag and suppressed duplicate notification.	**PASS**
TC-2.3	Item expiration date is exactly 4 days away.	Boundary	The Cron job scans the item but ignores it (outside strictly ≤ 3-day threshold).	Item scanned. No alert generated. Database state remained unchanged.	**PASS**

3.4.8. Feature 3: Smart Grocery List Management

Testing Objective: Validate the lifecycle loop between consuming an item and restocking it via the Grocery List.

Test ID	Scenario	Type	Expected Outcome	Actual Outcome	Status
TC-3.1	User manually decreases an inventory item's quantity to exactly `0`.	Happy Path	System automatically flags the item and moves it to the "My Grocery List" view.	Quantity reached 0. Item immediately appeared in Grocery List UI.	**PASS**
TC-3.2	User adds a "Custom Item" never previously in Core Inventory.	Edge Case	The item is added to the list successfully and behaves normally.	Custom item "Birthday Cake" added successfully and synced across roommate devices.	**PASS**
TC-3.3	User checks off a pending item in the Grocery List after purchase.	Happy Path	Item is removed from List and returned to Core Inventory with updated quantity/expiry.	Item checked off. Transition modal appeared. Item returned to active stock.	**PASS**

3.5. Requirements Inspection Checklist (Defect-Driven)

This checklist standardizes requirement reviews so the team can detect common defects early and record them consistently.

3.5.1. Purpose

Use this checklist to review requirement statements, identify defects, classify their severity, and assign follow-up actions.

3.5.2. How to Use

. Select the requirement section to review. . Check each requirement against the defect categories below. . Record findings in the review log. . Classify each issue as Critical, Major, or Minor. . Propose a concrete fix and assign an owner.

3.5.3. Severity Guide

* **Critical** — blocks implementation or testing, or may cause major incorrect behavior. * **Major** — creates significant ambiguity, inconsistency, or traceability problems. * **Minor** — reduces clarity or structure but does not block progress.

3.5.4. Defect-Driven Checklist

3.5.4.1. Omissions

* Is every requirement defined somewhere? * Are expected inputs, outputs, and system responses covered? * Are edge cases such as missing data, invalid values, or no response addressed?

3.5.4.2. Contradictions

* Does the requirement conflict with another requirement, goal, or constraint? * Are terms used consistently across documents?

3.5.4.3. Inadequacies

* Does the requirement reflect a real stakeholder need? * Is it valid for realistic student or roommate usage scenarios?

3.5.4.4. Ambiguities

* Can the statement be interpreted in more than one way? * Are vague terms such as "fast", "easy", or "soon" replaced with measurable criteria?

3.5.4.5. Unmeasurability

* Does the requirement include fit criteria such as time, threshold, count, or accuracy? * Is there a clear pass/fail condition?

3.5.4.6. Noise

* Is the statement relevant to system goals or constraints? * Would removing it change expected system behavior or quality?

3.5.4.7. Overspecification

* Does the requirement force a design decision too early? * Could the same goal be satisfied in more than one way?

3.5.4.8. Poor Structuring

* Is the requirement located in the correct section? * Is it mixed with assumptions, domain properties, or implementation details?

3.5.4.9. Opacity and Modifiability

* Are dependencies between requirements explicit? * Would changing this requirement force scattered edits across many documents?

3.5.5. Review Log Template

ID	Requirement Ref	Defect Type	Severity	Proposed Fix	Owner
RQA-001	REQ-XXX	Ambiguity	Major	Replace "quickly" with "under 2 seconds"	@username

3.5.6. Minimum Team Usage Rule

* Each team should apply this checklist at least once per sprint to active requirements. * Findings

should be linked to GitHub issues or PR comments for traceability.

4. Specific Topics

4.1. Stakeholder Identification

This section identifies stakeholders relevant to the domain and summarizes their goals, influence, and concerns.

4.1.1. 1. Primary Stakeholders (Direct Users)

These stakeholders interact with the system daily and their needs dictate the core functional requirements. They map directly to our defined user personas.

4.1.1.1. The Unofficial Household Manager (Persona: Andrea)

- **Who:** A frequent household supply buyer who naturally organizes the house and does bulk shopping.
- **Why:** Needs to keep the house perfectly stocked without carrying the entire mental load and wants to get reimbursed quickly for shared purchases.
- **How (Relationship):** Heavy, write-intensive usage. Creates shared inventories, logs bulk purchases, and sends reimbursement nudges.
- **Functional Needs:** Shared digital ledger, rapid data entry for bulk items, verifiable records of consumption, and low-stock alerts.

4.1.1.2. The Budget-Conscious Roommate (Persona: Sofia)

- **Who:** A student living in a shared apartment on a strict financial budget.
- **Why:** Needs to track shared groceries, split costs fairly, and completely avoid buying duplicate items due to limited funds and storage space.
- **How (Relationship):** Read/Write usage. Checks the app while at the store to verify current stock and logs personal vs. shared expenses.
- **Functional Needs:** Real-time inventory synchronization, duplicate purchase warnings, and distinct tracking of personal vs. shared household items.

4.1.1.3. The Busy Solo Student (Persona: Mateo)

- **Who:** A student living alone with severe time constraints and limited mental bandwidth for chores.
- **Why:** Needs to minimize time spent grocery shopping and avoid throwing away expired perishable food.
- **How (Relationship):** Notification-driven usage. Relies on the app to remind him of what he has before it goes bad.
- **Functional Needs:** Automated expiration alerts, simple UI for quick checks, and grocery list auto-generation based on low stock.

4.1.1.4. The Health-Conscious Student (Persona: Julian)

- **Who:** A student with strict dietary or medical needs who keeps their inventory separate from roommates.
- **Why:** Needs to maintain an uninterrupted supply of specialty items and track consumption rates of expensive supplements.
- **How (Relationship):** Precision usage. Sets exact thresholds for reordering.
- **Functional Needs:** Adjustable low-stock threshold alerts, strict permission/privacy boundaries for personal items within a shared household.

4.1.2. 2. Secondary & Indirect Stakeholders

These individuals affect or are affected by the system but use it sporadically or indirectly.

4.1.2.1. Transient Household Members (Guests, Former/Future Roommates)

- **Who:** People temporarily interacting with the household inventory.
- **Why:** Guests consume items without tracking ("shrinkage"); former roommates leave orphaned items; future roommates need to see current stock before moving in.
- **Relationship:** Cause inventory drift or require access modifications.
- **Functional Needs:** Ownership transfer protocols, easy user removal/invitation, and inventory reconciliation mechanisms for untracked consumption.

4.1.2.2. Parents / Family Contributors

- **Who:** Family members helping fund the student's living expenses.
- **Why:** Want to ensure the student has enough food and isn't wasting money on duplicates or expired goods.
- **Relationship:** Read-only or indirect financial contributors.
- **Functional Needs:** Shareable grocery lists or read-only access to household needs.

4.1.2.3. Grocery / Retail Stores

- **Who:** Supermarkets and wholesale clubs.
- **Why:** Provide the products and barcode data the system relies on.
- **Relationship:** Data originators.
- **Functional Needs:** Accurate barcode scanning API integrations and manual entry fallbacks for unrecognized store brands.

4.1.3. 3. External & Technical Stakeholders

4.1.3.1. Development Team & Database Maintainers (INSO4115 Team)

- **Who:** The software engineering students building and maintaining the app.

- **Why:** Need clear requirements, reliable backend uptime (Supabase), and clean architecture.
- **Functional Needs:** Well-documented APIs, robust database schema, automated backups, and CI/CD pipelines.

4.1.3.2. Project Evaluators (Project Managers / Course Staff)

- **Who:** Course instructors evaluating the software engineering process.
- **Why:** Assess the project against syllabus requirements and domain problem resolution.
- **Functional Needs:** End-to-end traceability, comprehensive AsciiDoc documentation, and functional completeness.

4.1.3.3. Mobile Platform Reviewers (App Store / Google Play)

- **Who:** Automated and manual reviewers for app stores.
- **Why:** Ensure the app complies with privacy, data handling, and UI guidelines.
- **Functional Needs:** Secure authentication, clear privacy policies, and crash-free navigation.

4.1.4. 4. Stakeholder Matrix (Power / Interest Grid)

Stakeholder	Interest	Influence	Management Strategy	System Module Mapping
Household Manager (Andrea)	High	High	**Manage Closely:** Prioritize their core needs (bulk entry, shared ledger) as their frustration causes household breakdown.	Shared Inventory, Ledgers
Budget/Solo Users (Sofia, Mateo)	High	Medium	**Keep Satisfied:** Ensure synchronization, duplicate warnings, and expiration alerts are highly reliable.	Expiration Alerts, Real-time Sync
Project Evaluators (Course Staff)	High	High	**Manage Closely:** Adhere strictly to project guidelines, milestone rubrics, and documentation traceability.	Documentation, Traceability
DB Maintainers / Dev Team	High	High	**Manage Closely:** Design robust architecture that prevents database bloating or security flaws.	DB Schema, Security Rules (RLS)
Transient Members (Guests, etc.)	Low	Medium	**Monitor:** Build robust error-handling and reconciliation tools to handle the inventory drift they cause.	Inventory Reconciliation, Invites

Stakeholder	Interest	Influence	Management Strategy	System Module Mapping
Parents / Guardians	Medium	Low	**Keep Informed:** Provide simple sharing features (like exporting a grocery list) without full app onboarding.	Grocery List Export
Platform Reviewers	Low	High	**Keep Satisfied:** Ensure strict adherence to platform UI and privacy guidelines to prevent launch blockers.	Authentication, App Settings

4.2. Personas

Personas represent key user/stakeholder types and are the single source of truth for all requirements, user stories, and features. Each persona includes goals, frustrations, behaviors, constraints, and a short scenario. The persona-to-needs trace table at the end of this file links each persona to specific needs (N1, N2, etc.), which are referenced throughout requirements for justification and traceability.

4.2.1. Persona 1: Mateo, The Busy Solo Student

- **Role:** Student living alone
- **Background:** Mateo is a full-time engineering student who also works a part-time job. He lives by himself in a small apartment near campus and has very little free time or mental energy for household management.
- **Goals:**
  - Minimize time spent grocery shopping.
  - Avoid throwing away expired perishable food.
  - Keep meal prep quick, simple, and predictable.
- **Frustrations:**
  - Frequently forgets what ingredients he already has in the fridge.
  - Buys fresh produce or meat, but it goes bad before he finds the time to cook it.
  - Experiences morning stress when he realizes he is completely out of coffee or milk.
- **Typical behaviors:**
  - Shops sporadically, usually only when the fridge is completely empty.
  - Relies entirely on his memory for his home inventory.
  - Orders expensive takeout when overwhelmed by a disorganized kitchen.
- **Constraints**:
  - Severe time constraints due to a heavy class and work schedule.

- Limited mental bandwidth for tracking chores; relies heavily on his smartphone for organization.

- **Scenario:** Mateo comes home late after a grueling day of classes. He remembers buying chicken breasts earlier in the week and plans to quickly pan-fry them for dinner. When he opens the fridge, he realizes he actually bought them almost a week and a half ago, and they have spoiled. Frustrated, he orders an expensive pizza delivery, breaking his weekly food budget, and later realizes he also has three open bottles of ketchup in the pantry. Success for Mateo looks like receiving an alert/notification (delivery TBD / in-app baseline) the day before his chicken expires, and being able to quickly check an app at the supermarket to confirm he doesn't need to buy a fourth bottle of ketchup.

4.2.2. Persona 2: Sofia, The Budget-Conscious Roommate

- **Role:** Student living with roommates and Budget-conscious student
- **Background:** Sofia lives in a shared apartment with two other students. She pays for her own living expenses using a mix of financial aid and a strict monthly budget.
- **Goals:**
  - Keep grocery and household supply costs as low as possible.
  - Ensure communal items like toilet paper and dish soap are split fairly.
  - Avoid buying any duplicates of items the apartment already has.
- **Frustrations:**
  - Roommates using her specific, more expensive dietary items without asking.
  - Running out of money before the end of the month due to poor collective household planning.
  - Wasting limited apartment storage space on duplicate bulk items.
- **Behaviors:**
  - Shops with a strict list and a calculator app open.
  - Tries to use a physical whiteboard on the fridge to track shared apartment needs, but her roommates forget to update it.
- **Constraints:**
  - Extremely limited financial budget.
  - Shared physical storage space that fills up quickly.
  - Reliant on uncooperative roommates to manually update the physical whiteboard.
- **Scenario:** Sofia goes to the supermarket with a strict $50 budget. She checks the apartment's shared whiteboard and sees they need toilet paper and dish soap. She buys them, using up a significant chunk of her cash. When she returns, she finds one of her roommates had already bought bulk toilet paper earlier that afternoon but forgot to erase it from the whiteboard. Now the apartment has no storage space left, and Sofia is short on cash for her own meals. Success for Sofia looks like a synchronized mobile app where changes are recorded and synchronized so everyone sees updated shared inventory and purchases, preventing her from spending her limited money on duplicate items.

4.2.3. Persona 3: Julian, The Health-Conscious Student

- **Role:** Student with dietary or medical needs
- **Background:** Julian is a student athlete who follows a strict, high-protein, gluten-free diet. He lives with roommates but keeps his food entirely separate due to his nutritional requirements and fear of cross-contamination.
- **Goals:**
  - Maintain a consistent, uninterrupted supply of specific, specialized dietary items.
  - Accurately track consumption rates of expensive supplements to budget for reorders.
- **Frustrations:**
  - Running out of core meal-prep ingredients mid-week.
  - Disorganized pantry spaces making it hard to quickly find what he needs.
  - Specialty items expiring because he bought too much in bulk to save money online.
- **Behaviors:**
  - Buys non-perishables in bulk online to save money.
  - Does heavy meal prep every Sunday afternoon.
  - Tracks his macronutrients religiously but struggles to translate that into a grocery inventory.
- **Constraints:**
  - Strict dietary policies that don't allow for substituting ingredients.
  - Limited local availability of his required brands.
- **Scenario:** Julian does his weekly meal prep on Sunday. Halfway through cooking, he goes to his pantry shelf to grab his special gluten-free pasta, only to realize he used the last box on Wednesday and forgot to order more. Because the local grocery store doesn't carry his brand, his entire meal plan is ruined. Success for Julian looks like threshold-based, adjustable low-stock alerts that remind him to reorder specialty items before he runs out.

4.2.4. Persona 4: Andrea, The Unofficial Household Manager

- **Role:** Frequent household supply buyer and Roommate who shares purchases
- **Background:** Andrea lives in a house with three other students. Because she is the most organized and has a car, she naturally fell into the role of the "unofficial house manager," doing the bulk of the household shopping at wholesale clubs.
- **Goals:**
  - Keep the house perfectly stocked with shared essentials without doing all the mental labor.
  - Get reimbursed quickly and accurately by roommates for shared purchases.
- **Frustrations:**
  - Constantly having to text the roommate group chat to ask what they are out of, getting no replies.
  - Getting stuck paying the bill when shared items are consumed but nobody admits to

finishing them.

- **Behaviors:**
  - Uses a chaotic mix of WhatsApp messages, scattered receipts, and Apple Notes to track house needs.
  - Frequently sends money requests to roommates after a shopping trip.
- **Constraints:**
  - Coordinates across three different and uncooperative roommate schedules.
  - Needs accurate, unarguable proof of consumption for financial reimbursements.
- **Scenario:** Andrea plans a trip to a wholesale club. She texts her roommates asking what they need, but only one replies. She guesses on the rest and buys shared snacks and cleaning supplies. The shared snacks disappear rapidly, and when she asks for everyone to split the cost, arguments break out over who actually ate them. Meanwhile, the house runs out of trash bags because Andrea assumed they had some hidden under the sink. Success for Andrea looks like a transparent, shared digital ledger where roommates can log what they finish and request items, making reimbursements unarguable and keeping the house perfectly stocked.

4.2.5. Persona-to-needs trace

Persona	Key needs
Mateo	Know what food is currently in stock; receive expiration alerts before items spoil; avoid running out of essential daily items
Sofia	Prevent duplicate purchases in shared households; split shared costs fairly among roommates; stay within a strict monthly budget
Julian	Maintain an uninterrupted supply of specialty dietary items; predict restocking needs using adjustable threshold alerts; keep personal items separate from shared inventory
Andrea	Coordinate shared household stocking without carrying all the mental load; obtain accurate, verifiable records for roommate reimbursement; know what is needed before a bulk shopping trip

4.3. Traceability

This section describes how traceability is established in the Digital Home Inventory project. Traceability means our deliverables are linked so we can answer questions about where requirements come from, which personas they serve, and how they connect to domain concepts, scenarios, and verification artifacts.

4.3.1. Overview

Traceability helps us understand relationships between different parts of our documentation. When something changes, we can see what else might be affected. For example, if a requirement is

modified, we can identify which personas and needs it supports, which user stories are impacted, and which scenarios/tests should be revisited.

Our approach focuses on creating clear links between artifacts without overcomplicating the documentation (Milestone 1 focuses on progress, not completeness).

4.3.2. Traceability approach (labels and links)

We maintain traceability through a simple labeling system:

4.3.2.1. Persona goals

Each persona's goals are labeled with a short code. (Goal descriptions are kept short to make them easy to reference.)

- **Mateo (M)**
- **MG1**: Know what I have without extra effort
- **MG2**: Avoid wasting expired food
- **MG3**: Avoid running out of daily essentials
- **Sofia (S)**
- **SG1**: Stay within a strict budget
- **SG2**: Split shared costs fairly
- **SG3**: Avoid duplicate purchases
- **Julian (J)**
- **JG1**: Avoid running out of specialty items
- **JG2**: Avoid waste from bulk purchases
- **JG3**: Keep personal items separate
- **Andrea (A)**
- **AG1**: Coordinate shared stocking without mental overload
- **AG2**: Get reimbursed accurately for shared purchases
- **AG3**: Know what is needed before bulk shopping

4.3.2.2. Needs requirements

Each requirement in Section 2.2 is tagged with which Needs IDs (N1–N5) it supports, and (when helpful) which persona goals it supports.

Example

- **IR-01 (Item Initialization)** → Needs: N1, N2 | Goals: MG1, JG1, AG3
- **IR-04 (Low-stock thresholds)** → Needs: N2 | Goals: MG3, JG1, AG1
- **IR-07 (Cost splitting)** → Needs: N5 | Goals: SG2, AG2

4.3.2.3. User stories requirements

Each user story in Section 2.2.1 lists which interface requirements it implements.

Example

- **US-INV-01** → Implements: IR-01, IR-02, IR-03, IR-05
- **US-ALT-01** → Implements: IR-04
- **US-SB-01** → Implements: IR-06, IR-07
- **US-RPT-01** → Implements: IR-08

4.3.2.4. Domain concepts requirements

Domain concepts from Section 2.1 (Terminology/Entities/Events/Functions) are linked to the requirements that depend on them.

Example

- **Item type / Item instance** → Appears in: IR-01, IR-02, IR-05
- **Household / Household member** → Appears in: IR-03, IR-06, IR-07
- **Low-stock threshold** → Appears in: IR-04
- **Expiration date** → Appears in: IR-05
- **Spending entry / Budget (optional scope)** → Appears in: IR-06, IR-07, IR-08

4.3.2.5. Scenarios/tests requirements (Milestone 1)

For Milestone 1, traceability is extended to the main validation and verification artifacts:

- Scenario IDs: **S1..Sx** (Section 3.2.1)
- Test case IDs: **TC-*** (Section 3.2.3)
- Walkthrough findings: **WF-*** (Section 3.2.2) once real sessions are performed

Example

- **S1 (Low-stock after consumption)** → IR-02, IR-04 → TC-ALT-01, TC-AUTO-01
- **S2 (Avoid duplicate purchase in shared household)** → IR-03, IR-08 → TC-INV-05, TC-RPT-01

4.3.3. Traceability matrix (Milestone 1 snapshot)

Goal	Need(s)	Requirement(s)	Story / Scenario	Domain concept(s)
MG1	N1, N2	IR-01, IR-02	US-INV-01 / S1	Item type, Item instance, Tracked inventory record
MG3	N2	IR-04	US-ALT-01 / S1	Low-stock threshold, Stock state
SG2	N5	IR-06, IR-07	US-SB-01	Spending entry, Shared vs personal

Goal	Need(s)	Requirement(s)	Story / Scenario	Domain concept(s)
JG1	N1, N2	IR-01, IR-04, IR-05	US-INV-01, US-ALT-01	Item instance, Expiration date, Threshold
AG2	N5	IR-06, IR-07	US-SB-01	Spending entry, Attribution
AG3	N3	IR-08	US-RPT-01 / S2	Dashboard metrics, Shared visibility

4.3.4. Questions traceability helps answer (kept concise)

- Which personas and needs are affected if a requirement changes?
- Which scenarios/tests must be revisited to verify the changed requirement?
- Which domain concepts are involved in a given requirement (to avoid terminology drift)?

4.3.5. Current traceability status (Milestone 1)

We have established traceability for:

- Persona goals: all personas have labeled goals (MG*, SG*, JG*, AG*).
- Needs: core needs N1–N5 are referenced by the interface requirements.
- Requirements: IR-01 through IR-08 are tagged to needs/goals and mapped to stories.
- Validation/verification: key scenarios and initial test cases are linked to the most central IRs.

NOTE	Deferred items for later milestones: * Additional edge-case interface requirements captured under “Deferred edge cases (Milestone 2+)” will be traced once they are promoted into explicit IR IDs. * Domain requirements (DR-*) will be added to the traceability matrix once the DR list is finalized and terminology is normalized across all requirements.

|Andrea |Coordinate shared household stocking without carrying all the mental load; obtain accurate, verifiable records for roommate reimbursement; know what is needed before a bulk shopping trip |===

4.4. Domain Requirements Acquisition

This section documents how raw domain evidence was gathered and how that evidence was later used during elicitation. To keep these activities distinct, this section separates requirements acquisition from requirements elicitation.

Requirements acquisition focuses on collecting raw domain material before it is interpreted. In this project, acquisition includes questionnaires, short interviews, contextual walkthroughs, and examples of current household tracking practices.

Requirements elicitation begins after acquisition. It includes grouping repeated problems,

identifying needs and conflicts, and transforming collected evidence into stories and requirements.

4.4.1. Questionnaire

To support requirements acquisition, the team prepared a short questionnaire aimed at collecting recurring household practices, coordination problems, and tracking pain points before moving into deeper interviews.

The questionnaire focuses on current behavior rather than desired features.

Sample questionnaire items:

* What kinds of household items do you most often lose track of? * How do you currently keep track of what you have at home? * Can you describe a recent time when you bought something you already had? * How do you usually tell that an item is running low? * Do you usually think in exact quantities or rough judgments like “almost empty” or “backup left”? * Do you ever store the same kind of item in multiple places? * Have you ever thrown something away because it expired or was forgotten? * If you live with others, how do you coordinate who buys shared items? * What kinds of reminders would be helpful, and what kinds would feel annoying? * What would make inventory tracking fast enough that you would actually keep using it?

4.4.2. Interview Protocol

The questionnaire is followed by short semi-structured interviews to obtain more specific stories and clarify ambiguous responses.

The protocol used in these sessions is:

#. assign a participant code before taking notes . begin with current tracking habits . ask for one or two recent concrete stories . probe for details such as item, location, what the participant believed, and what actually happened . ask about edge cases such as duplicate purchases, expired items, multiple locations, guests, and move-out situations . summarize the main points back to the participant to confirm understanding

Example prompts include:

* “Tell me about the last time you bought something and later realized you already had it.” * “Walk me through a specific time when you ran out of an item unexpectedly.” * “When you say an item was ‘almost empty’, what made you think that?” * “Can you describe a roommate situation that caused confusion about a shared item?”

4.4.3. Participant Coding

To preserve anonymity while still allowing statements from the same participant to be traced consistently across the document, participants are identified using stable codes.

Examples: * P01 — student living alone * P02 — student in shared apartment * P03 — student in shared apartment * P04 — student athlete * P05 — shared-household organizer

Quotes, notes, and walkthrough findings should refer to participant codes rather than real names.

4.4.4. From Acquisition to Elicitation

After acquisition, raw answers are reviewed and grouped into recurring domain problems and concepts. Those patterns are then used during elicitation to derive needs, stories, and requirements.

The transformation process used in this project is:

#. collect raw questionnaire and interview evidence . group repeated observations and pain points . identify the related domain concept or conflict . derive needs, candidate requirements, or open questions . link those results back to the source evidence

For example:

Source evidence	Observed pattern	Used later in
“If it takes more than a few seconds to log, I won’t do it.”	High-friction entry discourages continued use	Need N1, fast-capture requirements, US-INV-01
“If it is not on our fridge note, we assume we are out.”	Tracked understanding may diverge from reality	Domain concept analysis, inventory requirements
“We have rice in two places.”	The same item type may exist in multiple locations	Location-related requirements and terminology
“I want to remove access when someone moves out.”	Membership changes affect visibility and access	Shared-household requirements and stakeholder analysis

4.4.5. Acquisition Plan

This section defines how raw domain knowledge and evidence are gathered before they are interpreted into stakeholder needs and system requirements.

Requirements acquisition focuses on collecting raw observations, examples, and participant accounts. Elicitation happens later, when that material is interpreted into concepts, stories, and requirements.

4.4.5.1. Goals

- Capture real household practices instead of idealized descriptions.

- Understand shared-household coordination, including visibility, duplicates, ownership, and cost splitting.
- Identify friction points that cause tracking methods to be abandoned.
- Clarify important edge cases such as bulk purchases, multiple locations, and missing expiration information.
- Build a traceable base of raw domain evidence for later elicitation.

4.4.5.2. Methods

The following methods are part of requirements acquisition because they gather raw domain data before formal analysis or requirement extraction takes place:

- **Questionnaire:** short set of questions used to collect recurring household practices and pain points across participants.
- **Interviews (4–6):** semi-structured interviews with solo students and students in shared households, including at least one student athlete.
- **Contextual walkthroughs:** participants demonstrate their current tracking methods and explain their shorthand and terminology.
- **Item analysis:** review of receipts, pantry or fridge photos, and shared shopping messages to understand how items are tracked or forgotten in practice.
- **Light competitive scan:** review a small set of existing apps to identify repeated friction points such as entry effort and notification fatigue.

NOTE	Broader surveys and larger quantitative validation may be deferred if time is limited. This milestone prioritizes qualitative evidence that supports domain understanding and early requirements work.

4.4.5.3. Schedule

Timeframe	Activities
Late Jan. – Early Feb.	Conduct questionnaires, initial interviews, and contextual walkthroughs; expand the rough sketch with concrete quotes and examples.
Mid-Feb.	Review collected evidence, refine terminology, and record open questions and potential clashes.
Late Feb.	Use follow-up questions to clarify gaps; transform acquired evidence into personas, scenarios, stories, and core requirements.

4.4.5.4. Risks and Mitigations

- **Risk:** Participants are too busy for long interviews.
  - **Mitigation:** Keep sessions short and allow asynchronous follow-up.
- **Risk:** Participants describe ideal behavior instead of actual behavior.

    - **Mitigation:** Ask for recent concrete examples and current artifacts such as notes, chats, or receipts.
- **Risk:** Manual-entry tolerance is very low.
    - **Mitigation:** Ask explicitly what minimum information participants are willing to record.
- **Risk:** Acquisition and elicitation are mixed together too early.
    - **Mitigation:** Record raw evidence first, then interpret it later in a separate elicitation step.

4.4.6. Resources

This subsection lists supporting materials and evidence sources used during requirements acquisition. These resources support the acquisition process, but they are not a substitute for the questionnaire, interview protocol, or participant-coded raw evidence.

4.4.6.1. External Sources (selected)

- Household food waste research: supports the importance of visibility and expiration awareness in student households.
- Personal Information Management (PIM) research: supports the risk that high-effort updates and excessive reminders lead to abandonment.
- Household finance and budgeting research: supports the importance of spending awareness for budget-constrained students.

Extended literature notes: `docs/lecture-topic-tasks/domain-acquisition-ltt.adoc`

4.4.6.2. Internal Artifacts

- Domain rough sketch and narrative: `docs/2-descriptive/domain-description/rough-sketch/` and `docs/2-descriptive/domain-description/narrative/`
- Terminology table: `docs/2-descriptive/domain-description/terminology/terminology.adoc`
- Domain concept analysis: Section 3.1
- Validation and verification artifacts: Section 3.4
- Acceptance and E2E testing artifacts: Section 3.5

4.4.6.3. Notes for Completion

- Participant-coded interview and walkthrough findings should be linked from this section once finalized.
- Competitive scan results should be summarized here once completed.
- Acquisition completeness should be reviewed by checking whether all major needs and requirement areas are supported by raw evidence.

4.5. Inconsistency Management

An exploration of our domain terminology, database schema, events, actions and behaviors, shared

inventory definition, and duplicate item criteria led to the discovery of two inconsistencies. * **Event naming clash — "An Inventory has been generated":** This event, triggered by the addition of an item to the inventory for the first time, was described as changing the inventory so that it now reflects the presence of a new item. However, the term *Inventory* is defined in our terminology as "the list of available products in a home," while *Item* (a synonym for *Stock*) is defined as "an instance of a product." The event description created a designation clash with the existing glossary: the name alluded to the creation of an inventory, but the described change was a modification of an already-existing inventory. To resolve this clash, the event was split into two: **The "What changed" description of "An Inventory has been generated" was updated to: "The inventory is created and reflects the presence of the new item." This separates the creation of the inventory from the modification of an existing inventory, and clarifies that the inventory is created as a direct result of adding an item for the first time.** A new event, **"A new Product has been generated"**, was introduced for cases where an item is added to the inventory with no relation to an existing product. This event records the change to the inventory reflecting the new product description as well as the item count and description.

- **Ownership multiplicity mismatch — Group-Owned Inventory model:** The Group-Owned Inventory model in the Shared Inventory Definition document describes how all items can be mapped as belonging to the household, with multiple members able to add, edit, and delete them. It further describes that inventories have multiple stock entries, and that stock entries may optionally carry linked owner tags. This implies that items can have multiple owners. However, the ER and table diagrams for database item representation did not reflect this: the model was only satisfiable when items had exactly one owner.
    - To resolve this weak conflict, the ER and table diagrams for database item representation were updated. The relationship between items and owners was changed to a many-to-many relationship with total participation on both sides. A new table, **Owns**, was added to represent this relationship. This change allows an item to be owned by multiple members while preserving the ability of a member to own multiple items.

Generative AI Disclosure

In our Digital Home Inventory project, we utilized generative AI tools to enhance our workflow while maintaining ownership of the core ideas and decisions. The tools we used were Claude 4.5 (accessed via a free account), ChatGPT (free tier), and DeepSeek AI. These were primarily employed during the brainstorming and structuring phases, where they helped generate initial ideas for project features, such as inventory tracking, notifications for low stock or expirations, budget management, roommate sharing to avoid duplicates, and report generation. For documentation, we used AI to check grammar, suggest edits, and refine the structure to ensure it correlated correctly with rubric criteria, ensuring the documentation met the assignment criteria. The core work originated from our team, starting with original components like our feature analysis, research on requirements (e.g., Supabase schema for items with privacy fields, Flutter UI for scanning), and project scope discussions. AI was used to enhance these, providing suggestions on implementation steps or alternative structures, but we always edited and revised the output to align with our vision. For instance, when AI suggested feature requirements like API integrations for expiration dates, we reviewed them against the A3 document and team discussions, rejecting or modifying those that didn’t fit our student-focused goals. To verify AI-generated content, we fact-checked everything with the team, cross-referencing course materials, rubrics, and real-world examples from similar

apps. If inconsistencies arose, such as AI recommendations that conflicted with our privacy feature for personal items, we resolved them through brainstorming sessions, deciding collectively whether to incorporate, edit, or discard. One example: AI proposed a complex budgeting algorithm, but it didn't align with our simple student needs, so we simplified it after team review. From using AI, we learned it streamlined research and idea generation, freeing up time for deeper reflection on what the app truly needed for student users. It didn't reduce engagement, instead it forced us to critically evaluate outputs, enhancing our learning and analytical skills. Overall, AI acted as a supportive tool, not a replacement, helping us iterate faster while keeping the project authentic to our group's efforts.